蓝牙热敏打印机

开发教程

——从原理图到 App 联调的完整实践——

适用硬件：ESP32-WROOM-32E + JX-2R-01/JX-700-48 兼容打印头

软件框架：Arduino-ESP32 2.0.14（PlatformIO）

通信方式：经典蓝牙 SPP，兼容喵喵机公版 App

文档版本：v1.0

编写日期：2025-11

本教程基于项目资料中的示例代码、原理图与说明文档整理，
并补充了原理推导、参数计算、调试方法与常见故障对照。

目　　录

前言 6
这本教程解决什么问题 6
读者对象与前置知识 6
约定 6
第 0 章 项目总览 7
0.1 这台机器能做什么 7
0.2 系统组成与信号流 8
0.3 三条学习路线 10
0.4 物料清单（BOM） 10
0.5 工具与软件清单 11
0.6 30 分钟最小可行实验 11
第 1 章 热敏打印原理 13
1.1 热敏纸为什么会变黑 13
1.2 打印头的结构 13
1.3 一行数据的电学模型 14
1.4 加热时间与打印浓度 15
1.5 分辨率、点距与走纸 15
1.6 影响打印质量的八个因素 16
1.7 保护打印头的五条铁律 16
第 2 章 硬件设计详解 17
2.1 电源树：一台打印机的骨架 17
2.2 主控最小系统 18
2.3 打印头接口 18
2.4 升压电路（VH 8.4V） 20
2.5 电机驱动 20
2.6 缺纸检测 21
2.7 打印头温度检测 22
2.8 电池电压采样 22

2.9 充电管理 23
2.10 软开关机 24
2.11 USB、串口与自动下载 24
2.12 按键、LED 与蜂鸣器 24
2.13 设计校核：四个必须算的数 25
2.14 PCB 布局清单 25
2.15 打样、焊接与上电自检 25
第 3 章 开发环境搭建 27
3.1 三种开发方式怎么选 27
3.2 安装 VSCode 27
3.3 安装 PlatformIO 27
3.4 创建第一个工程 28
3.5 platformio.ini 完整配置 28
3.6 编译、烧录、看日志 29
3.7 串口调试的正确姿势 30
3.8 常见编译错误对照表 30
3.9 工程组织建议 31
第 4 章 ESP32 基础 32
4.1 这颗芯片到底有什么 32
4.2 GPIO：34 个脚不是都能随便用 32
4.3 按键与去抖 34
4.4 时间：millis、micros、delay 34
4.5 中断 35
4.6 串口 UART 35
4.7 SPI 35
4.8 ADC 36
4.9 PWM 与蜂鸣器 36
4.10 看门狗与复位原因 37
4.11 FreeRTOS：为什么需要两个任务 37
4.12 蓝牙栈：经典 BT 与 BLE 的区别 38

第 5 章 打印头驱动 39
5.1 一颗打印头里有什么 39
5.2 时序：四个时间参数 40
5.3 数据打包：384 点怎么变成 48 字节 42
5.4 sendData：把一行送进去 42
5.5 加热控制：为什么要"分段" 43
5.6 完整打印函数 44
5.7 从图片到点阵：完整链路 44
5.8 打印质量调优 45
5.9 三条保命守则 46
第 6 章 步进电机与走纸 47
6.1 两相四线步进电机 47
6.2 八拍与四拍 48
6.3 力矩、速度与丢步 48
6.4 走纸分辨率：必须自己标定 49
6.5 走纸的机械细节 49
6.6 电机噪声与发热 49
第 7 章 蓝牙通信 50
7.1 为什么是"经典蓝牙 SPP"，不是 BLE 50
7.2 BluetoothSerial 的完整 API 50
7.3 设备名、可见性与配对 51
7.4 事件回调：知道对方连上了没 51
7.5 缓冲：60KB 的天花板 52
7.6 收发模型：别用 `while(available())` 52
7.7 连接稳定性 53
7.8 实测吞吐 53
7.9 调试工具 54
第 8 章 通信协议（核心） 55
8.1 协议全貌 55
8.2 CRC32：两个不同的覆盖范围（最坑的地方） 55

8.3 ACK 帧 56
8.4 指令表（2.0.3 版，47 条） 56
8.5 握手：App 连上后做了什么 58
8.6 打印流程 59
8.7 状态机实现（可直接用） 60
8.8 应答实现 61
8.9 常见故障对照 61
8.10 自己写上位机 62
第 9 章 固件整合 63
9.1 分层架构 63
9.2 config.h：所有"魔法数字"的家 63
9.3 main.cpp：一个能跑的主循环 64
9.4 打印任务：队列 + 任务 65
9.5 自动关机 65
9.6 看门狗与异常 66
9.7 固件升级的两条路 66
第 10 章 联调与测试 67
10.1 分阶段测试计划 67
10.2 打印测试页 67
10.3 时序验证 68
10.4 电源完整性 68
10.5 长时间稳定性 68
10.6 常见问题总表 68
10.7 性能数据（实测参考） 69
附录 71
附录 A 引脚总表 71
附录 B 指令速查卡 71
附录 C 物料清单（BOM） 72
附录 D 术语表 72
附录 E 参考资料 73

附录 F 出厂前检查清单..... 73

前言

这本教程解决什么问题

随附的《蓝牙打印机开发资料》用 45 页把整台机器讲了一遍，但它是"资料"而不是"教材"：代码是片段式的，硬件只有原理图截图，很多"为什么这样做"没有展开，还有一些笔误和前后不一致的地方。很多人照着做，能点亮 LED，却在"打印出来是白的""打一半卡住""App 连上但一直转圈"这几步卡住好几天。

这本教程的目标只有一个：**让你从零把这台打印机做出来，并且知道自己每一步在做什么**。为此它做了三件事：

1. **补齐原理链**：热敏纸为什么会变黑、一行 384 个点怎么变成 48 个字节、为什么不能 384 个点同时加热、蓝牙 SPP 的缓冲区为什么必须改。知道了这些，出问题你才能定位。
2. **给全代码**：示例工程里没有 `platformio.ini`，没有可用的接收缓冲区方案，打印函数里有几个隐藏的越界和 off-by-one。本教程给出可编译、可运行的完整版本，并逐段讲解。
3. **建立调试方法**：从"电源是否正常"到"字节流是否正确"，按层给出现象-原因-验证手段的对照表，让你不再靠猜。

读者对象与前置知识

你是	建议读法
完全新手，只想做出一台能打印的机器	第 0 章 → 第 3 章 → 第 5 章 → 第 8 章 → 第 10 章
会单片机，想搞懂热敏打印头的驱动	第 1 章 → 第 5 章 → 第 6 章 → 第 8 章
想自己画板、改硬件	第 2 章 → 第 6 章 → 第 10 章
想做产品、要接自己的 App	第 7 章 → 第 8 章 → 第 9 章 → 第 10.5 节

需要的前置知识只有三条：看得懂 C 语言（指针、结构体、位运算）、知道什么是 GPIO 和串口、会用万用表。其余全部在文中解释。

约定

- 文中 `0x02` 这种写法表示十六进制字节；`0b1010` 表示二进制。
- "高位在前（MSB first）"指一个字节里 bit7 先发出，这是 SPI 与打印头共同的约定。
- 代码中的引脚号一律以**示例工程的头文件为准**，与原理图丝印可能有出入，第 0.2 节给了一张总表。
- 标注"⚠ 原资料有误"的地方，是本教程对随附资料的纠正，请以本教程为准。

第 0 章 项目总览

0.1 这台机器能做什么

它是一台 58mm 规格的便携热敏打印机：手机通过蓝牙连上它，把文字、图片、二维码发过去，它一行一行地加热热敏纸，把内容"烧"出来。整机由一节 18650 锂电池供电，插上 Type-C 就能充电，也能当串口给 ESP32 下载程序。

项目	参数	说明
打印方式	行式热敏打印	直接热敏，无需墨盒色带
打印宽度	48mm / 384 点	每行固定 384 个发热点
分辨率	203dpi（8 点/mm）	点距 0.125mm，行高 0.125mm
打印头	JX-2R-01（兼容 JX-700-48）	6 段 STB，内置热敏电阻
走纸	两相四线步进电机	每步约 0.0625mm（以实测标定为准）
主控	ESP32-WROOM-32E	240MHz 双核，4MB Flash
通信	经典蓝牙 SPP（串口透传）	设备名 `MaoPaperang`
供电	18650 锂电池 3.7~4.2V	Type-C 5V 充电，500mA
打印头电源	升压至 8.4V（VH）	峰值电流 1.5~2A
传感器	缺纸检测、打印头温度、电池电压	三个 ADC/GPIO 通道
交互	1 个按键、1 个 LED、蜂鸣器	开机 / 走纸 / 状态指示

0.2 系统组成与信号流

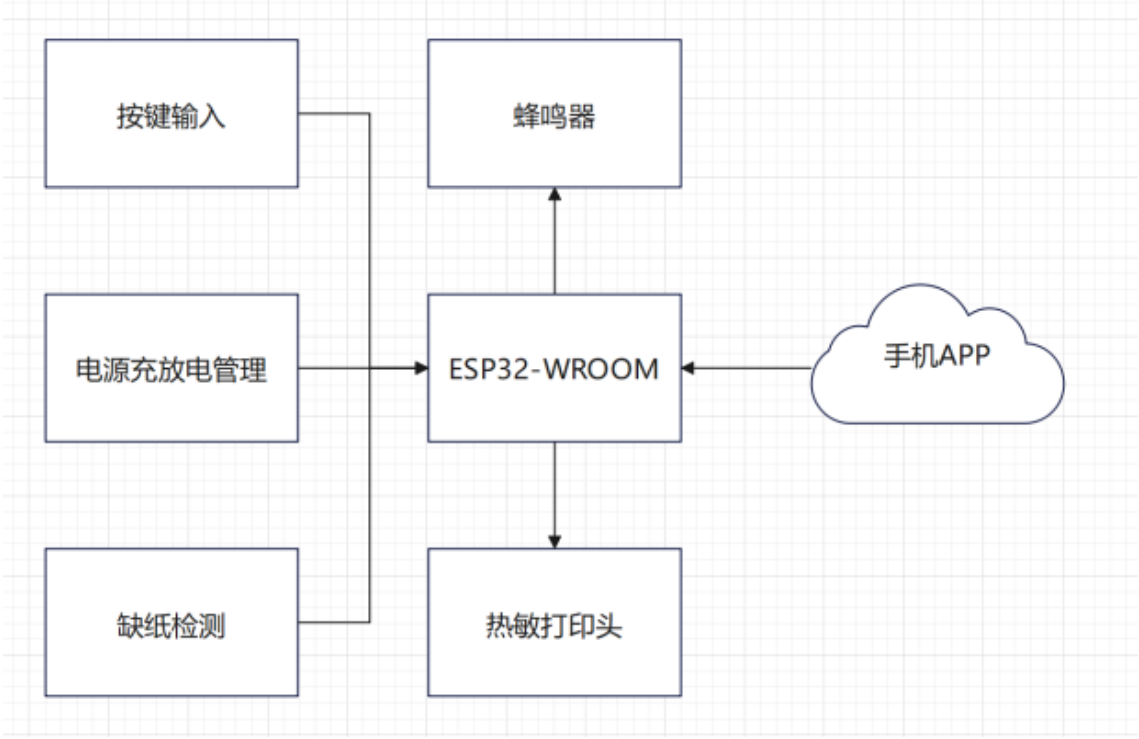


图 0-2 原资料给出的硬件设计框图（p03）：主控、电源、打印头与电机的连接关系

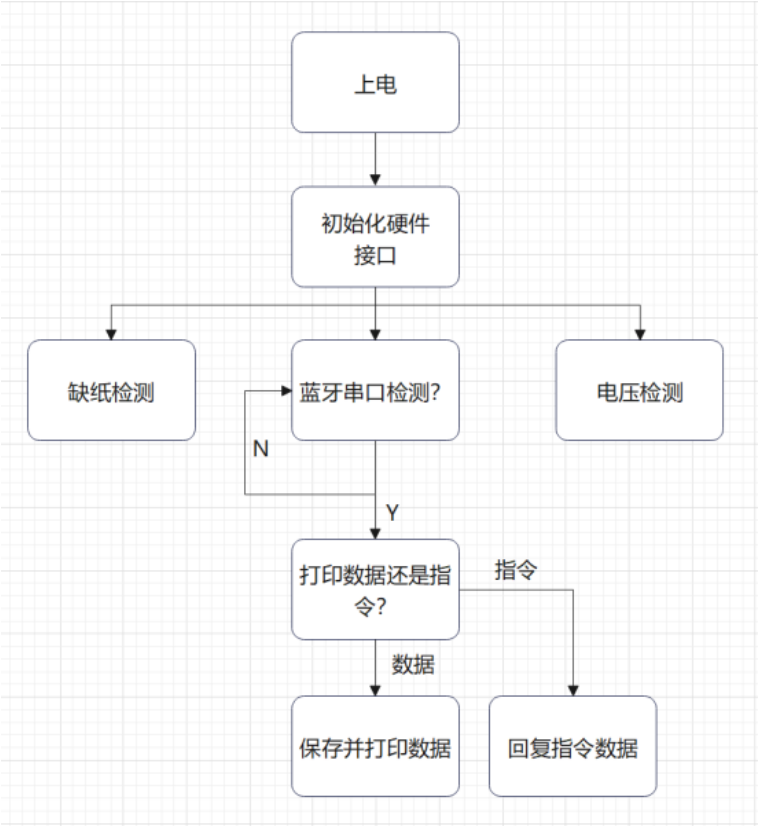


图 0-3 原资料的软件设计流程图（p04）：初始化、蓝牙收包、打印调度

图 0-1 蓝牙热敏打印机系统框图

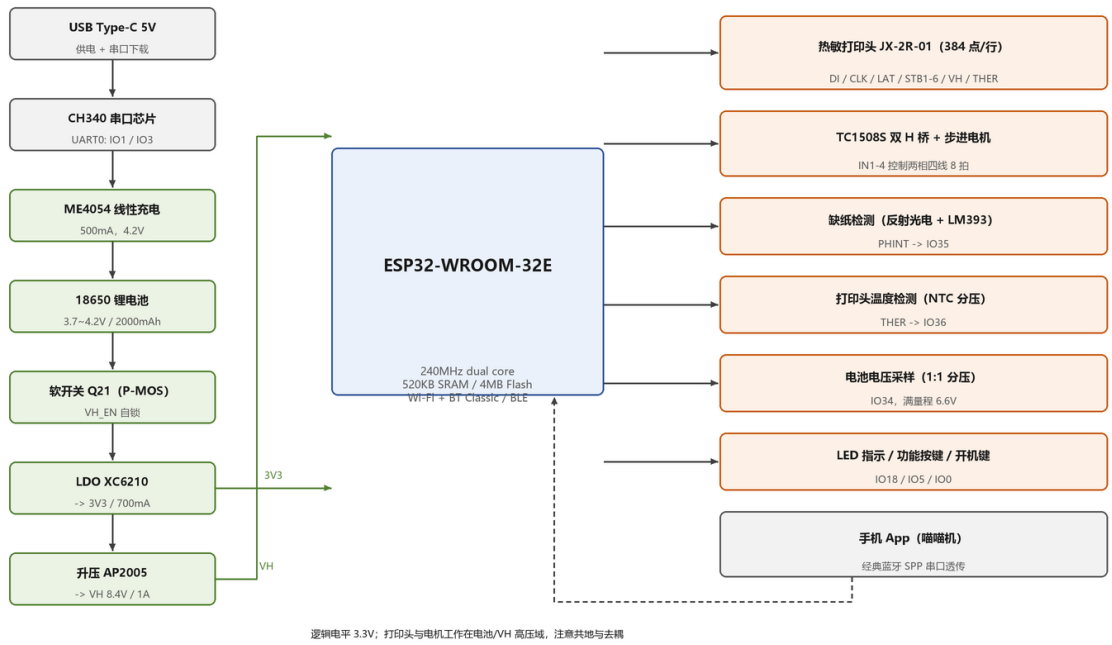


图 0-1 系统框图：电源域、主控与四个外设通道

数据是这样流动的：

```
手机 App ──蓝牙 SPP──> ESP32 串口缓冲 ──状态机解析──> 位图缓冲(48B/行)
                                                    ┌──────────────┴──────────────┐
                                                    ▼                             ▼
                                          SPI 送 48 字节到打印头          每行结束走一步电机
                                                    │                             │
                                          LAT 锁存 → STB 选通加热          热敏纸前进 0.125mm
```

一次完整的打印由三条并行的"时间线"共同决定成败：**数据线**（蓝牙收包 → 解析 → 组帧）、**加热线**（SPI 送数 → 锁存 → 选通 → 冷却）、**机械线**（电机走纸 → 压纸轮 → 热敏纸）。三条线任何一条慢半拍，就会出现条纹、错位或打印变浅。

引脚总表（以示例代码为准）

功能	ESP32 引脚	方向	备注
打印头 DI（数据）	IO13	输出	SPI MOSI
打印头 CLK（时钟）	IO15	输出	SPI SCK，1MHz
打印头 LAT（锁存）	IO17	输出	低电平 1μs 后拉高锁存
打印头 STB1~6	IO14 / IO27 / IO26 / IO25 / IO33 / IO32	输出	高电平加热
打印头电源使能	IO28（VH_EN）	输出	同时兼作整机自锁
电机 A- / A+	IO23 / IO22	输出	接 TC1508S IN1 / IN2
电机 B- / B+	IO21 / IO19	输出	接 TC1508S IN3 / IN4

功能	ESP32 引脚	方向	备注
缺纸检测 PHINT	IO35	输入	只输入引脚，无内部上拉
打印头温度 THER	IO36	模拟输入	ADC1_CH0
电池电压	IO34	模拟输入	ADC1_CH6，1:1 分压
LED	IO18	输出	低电平点亮
功能按键	IO5	输入	上拉，按下为低
BOOT 按键	IO0	输入	下载模式
串口 TX / RX	IO1 / IO3	—	接 CH340

> ⚠ 注意：IO34/35/36/39 属于 ESP32 的**只输入**引脚，没有内部上拉/下拉，也不能输出。把它们当普通 GPIO 用 `digitalWrite` 是无效的，这是新手最常见的坑之一。

0.3 三条学习路线

路线	你要做的事	时间
A. 纯软件（推荐先走）	拿现成板子，只改固件：跑通打印、接自己的 App	1~2 天
B. 软硬全栈	打板、焊板、写固件、调试到能卖	2~4 周
C. 只做硬件	电源、充电、升压、电机驱动，固件用现成的	1~2 周

0.4 物料清单（BOM）

核心器件

位号	型号	作用	关键参数
U20	ESP32-WROOM-32E	主控	4MB Flash，板载天线
U21	TC1508S	双 H 桥电机驱动	两通道，逻辑 3.3V 兼容
U22	ME4054BM5G-N	锂电池线性充电	500mA，4.2V 浮充
U23	AP2005SPER	升压 DC-DC	3.7V → 8.4V
U24	XC6210B332MR	LDO 稳压	3.3V / 700mA
U25	Type-C 16P 母座	供电与下载	5V/2A
U26	USB 转串口（CH340C 类）	下载与调试	内置晶振
U29	LM393	双比较器	缺纸与温度整形
Q21	AO4407A	P-MOS 电源开关	-30V，软开关自锁
Q23	AO3400A	N-MOS 栅极驱动	30V
D20	SS54	升压续流	40V / 5A 肖特基

位号	型号	作用	关键参数
L20	4.7μH 功率电感	升压储能	饱和电流 ≥3A
FPC21	32Pin 0.5mm 座	打印头排线	X10B25L30T
BT1	18650 电池	供电	2000mAh 以上，带保护板

被动件要点

- 输入/输出电容：升压输入端至少 22μF，输出端 47μF + 100nF，否则打印瞬间电压跌落会直接表现为"打印变浅"。
- 去耦：ESP32 的 3V3 脚旁必须有 0.1μF，模组电源脚再加 22μF。
- 限流/分压电阻精度建议 1%，NTC 分压电阻与 NTC 同精度，否则温度换算误差会超过 5℃。

0.5 工具与软件清单

类别	推荐	用途
IDE	VSCode + PlatformIO	编译、下载、串口监视
串口工具	任意串口助手 / `vofa+`	看日志、画波形、看图片数据
取模软件	PCtoLCD2002	汉字/图片转点阵
图像处理	画图（Windows）或 Python + Pillow	生成 384 宽单色位图
万用表	必须	量 VH、电池、电流
示波器/逻辑分析仪	强烈建议（≥24MHz 采样）	看 SPI 时序、STB 脉宽
可调电源	建议	限流 0.5A 首次上电
蓝牙调试	喵喵机 App（仅安卓）或 nRF Connect	验证连接

0.6 30 分钟最小可行实验

不要一上来就焊整板。按下面顺序，每一步都"看得见结果"再往下走：

第 1 步：只让 ESP32 活着。 用 USB 线供电，串口每 2 秒打印一次 `alive`，板载 LED 闪烁。这一步验证：供电、下载电路、串口驱动、编译链。

第 2 步：点亮打印头的 LED 指示灯（如果有）并测 VH。 万用表量升压输出，应在 8.4V±0.3V。如果只有 3.7V，说明升压没工作——先别接打印头。

第 3 步：让电机转。 写一个空循环，8 拍相序翻转，听声音、看轴转。转不动先查 TC1508S 的 VDD 和 IN1~4。

第 4 步：打一条黑边。 把 200 行全部填 `0xFF`，走一遍打印流程。出来一条连续的黑色带 → 打印头、SPI、时序、电源全部正确。

第 5 步：打一张测试图。 用取模软件生成 384×200 的图案，验证位序、行对齐、走纸比例。

第 6 步：接蓝牙。 手机搜到 `MaoPaperang`，用串口助手发几个字节，ESP32 回显。

第 7 步：跑协议。 用本教程附带的 Python 上位机脚本握手 + 打印，最后再用喵喵机 App 验证兼容性。

第 1 章 热敏打印原理

1.1 热敏纸为什么会变黑

热敏纸表面涂了一层"隐形墨水"：**隐色染料**（无色，通常是结晶紫内酯类）、**显影剂**（双酚 A 或双酚 S）和**敏化剂**，三者分散在树脂黏结剂里。常温下染料与显影剂是分开的固体颗粒，所以纸是白的。

当局部温度升到大约 **90~120℃** 并维持几毫秒，敏化剂先熔融，染料与显影剂在熔融相里接触、发生开环反应，生成有色的内酯结构，颜色就固定下来了。整个过程是**物理+化学**的，不需要水墨，但需要三样东西同时到位：**足够的热量、足够的时间、足够的压力**。

一条重要推论：**热敏纸对能量是积分型的**。同样的能量，高温短时和低温长时出来的黑度不同——这就是为什么"加热时间"必须跟着"电源电压"和"打印头温度"动态调整，否则电池从 4.2V 掉到 3.5V 时，打印会肉眼可见地变浅。

1.2 打印头的结构

打印头（JX-2R-01 / JX-700-48）是把"一排电阻"做在陶瓷基板上：

- **384 个发热点**排成一条直线，总长 48mm，点距 0.125mm，对应 203dpi。
- 每个点是一个薄膜电阻，阻值典型 150~200Ω（以规格书/实测为准），下面垫着釉层，上面覆盖耐磨层。
- 每个点前面串了一个**驱动门**：门的一端接 VH（8.4V），另一端接 GND，门的开关由"数据位 AND STB 选通"决定。
- 数据用**移位寄存器**串行灌入：384 位 = 48 字节，分 6 片 64 位驱动 IC 级联。
- 灌完之后用 **LAT** 一次性锁存到并行输出端，此时 384 个门都"待命"，但还没通电。
- 再用 **STB1~STB6** 分别（或同时）打开对应 64 点的门，电流流过电阻，发热。

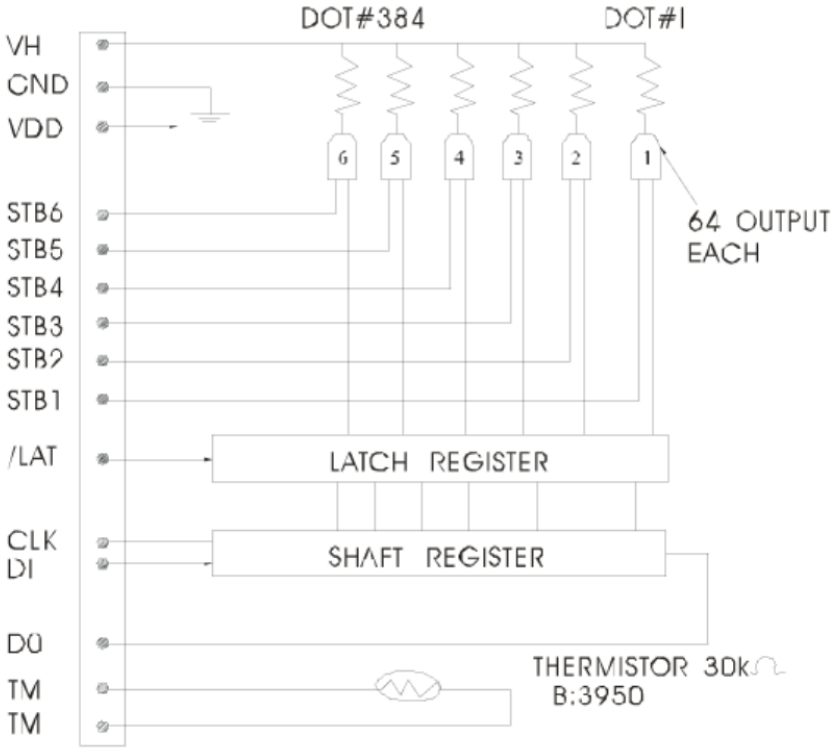


图 1-1 打印头内部电路示意（原资料 p23）：移位寄存器 + 锁存器 + 与门 + 功率管

所以打印一行的动作可以精确地写成五步：

```
① 移位：CLK 打 48 个脉冲，DI 上按 MSB 顺序送 48 字节
② 锁存：LAT 拉低 ≥1μs 再拉高，384 位数据进入并行输出
③ 选通：把要打印的那几段 STB 拉高（低有效则为拉低），开始加热
④ 保持：维持 t 微秒（由浓度、电压、温度决定），然后关断 STB
⑤ 走纸：电机走一步（0.125mm），回到 ①
```

> ⚠ 第 ④ 步和第 ⑤ 步的顺序不能颠倒。先关 STB 再走纸，否则打印头在走纸过程中一直通电，轻则拖影，重则烧毁发热点。

1.3 一行数据的电学模型

这一节决定了你的电源设计能不能扛住。

设单点电阻 R = 176Ω（典型值），打印头电压 VH = 8.4V：

量	公式	数值
单点电流	I = V/R	47.7mA
单点功率	P = V²/R	0.40W
一段（64 点）全黑	64 × I	3.05A / 19.6W
全行（384 点）全黑	384 × I	18.3A / 154W
实际平均黑点率 20%	77 点	3.7A / 30.8W

结论：绝对不能让 384 个点同时加热。 18A 会把电池、升压、FPC 排线全部拉垮，电压瞬间跌到 6V 以下，结果是"打出来一片灰"甚至复位。

工程上有三种限流手段，通常组合使用：

1. **分段选通**：6 段 STB 轮流打开，同一时刻最多 64 点。峰值电流降到 3A，代价是每行要 6 个时间片。
2. **限制单行黑点数**：统计一行里"1"的个数，超过阈值（比如 128 点）就自动延长加热时间但降低占空比，或把这一行拆成两次打。
3. **硬件限流**：升压芯片的限流点、电池保护板的过流点、串联小电阻，都能兜底。

一份可用的功率预算（VH 8.4V、电池 3.7V、升压效率 85%）：

场景	电池端电流	说明
待机 + 蓝牙广播	40mA	主要消耗在射频
电机走纸	250~400mA	两相同时导通
打印一行（20% 黑，分 6 段）	1.2~1.6A 峰值	与电机错开相位
最坏情况（连续黑块）	3A 以上，需固件干预	否则触发保护

1.4 加热时间与打印浓度

打印头的规格书会给一条"加热脉冲宽度 vs 黑度"的曲线。实用近似是：

```
t_heat = t_base × (8.4 / VH)² × k_temp × (density / 64)
```

- `t_base`：额定条件（8.4V、25℃、浓度 64）下的基准时间，典型 0.8~1.0ms。
- `(8.4/VH)²`：功率与电压平方成正比，电压低了要按平方补回来。
- `k_temp`：打印头温度补偿系数，冷的时候（<10℃）取 1.1~1.3，热的时候（>40℃）取 0.8~0.9。
- `density/64`：用户可调的浓度档位，直接线性缩放。

示例代码里 `HEAT_TIME = 5000μs` 是一个**很保守**的值（远大于 800μs），好处是在 5V 供电、劣质纸上也能出黑，坏处是慢、烫、费电。调试时建议先用 5000 保证能出图，再逐步降到 1000~1500 观察黑度，最终按上面的公式做动态补偿。

> 不要用"看起来够黑"当唯一标准。用同一张图在 4.2V / 3.7V / 3.4V 三种电量下各打一遍，黑度差超过一档就说明补偿没做好。

1.5 分辨率、点距与走纸

概念	数值	关系
水平分辨率	8 点/mm（203dpi）	384 点 = 48mm
点距	0.125mm	相邻发热点中心距

概念	数值	关系
行高	0.125mm	走纸一步的距离
电机步距	约 0.0625mm/步	半步模式下 2 步 = 1 行
一行时间	约 5~10ms	决定打印速度
打印速度	12~25mm/s	行高 ÷ 行时间

关键点：走纸量必须等于行高，否则图像会被拉长或压扁。 示例代码每行调用一次 `goFront_oneRow(500)`，即走 8 个半步。如果你的电机减速比、轮径和示例不同，必须重新标定：

1. 打印一张 100 行的纯黑条（100 × 0.125mm = 12.5mm）。
2. 用卡尺量实际长度 L。
3. 比例 = L / 12.5。若 L=25mm，说明每行走多了 1 倍，把每行的步数减半或把 `delayMicroseconds` 减半。

> ⚠ 原资料的"每步 0.0625mm"与示例代码"每行 8 步"放在一起会得出"每行 0.5mm"，与 203dpi 的 0.125mm 矛盾。原因多半是"半步/整步"和"减速比"的口径不同。**以实测标定为准**，不要照抄数字。

1.6 影响打印质量的八个因素

因素	现象	对策
VH 电压跌落	整体偏浅、右边更浅	加大输出电容、缩短走线、分段加热
加热时间不足	灰、字发虚	提高 `t_base`，做电压补偿
加热过度	字糊、边缘扩散、纸卷曲	降时间，限黑点率
打印头积碳/脏污	竖向白条	无水乙醇棉签清洁（断电、冷却后）
压纸轮压力不均	一侧浅	检查弹簧、导轨平行度
走纸丢步	图像被压扁、重影	降速、提高电机电流、加续流
热敏纸质量差	发色不足、保存褪色	换纸，提高浓度
打印头温度过高	越打越浅（保护性降功率）	降速、间歇打印、加散热

1.7 保护打印头的五条铁律

1. **无纸不许加热**：缺纸时热敏纸不吸热，热量全部积在发热点上，几秒钟就能烧出永久白点。缺纸信号一出现，立刻拉低所有 STB 并关 VH。
2. **STB 不许常开**：任何情况下 STB 的占空比都要受控。调试时如果程序跑飞，先断电。
3. **换纸/装纸必须断电**：FPC 带电插拔会打火，击穿驱动 IC。
4. **清洁只能用无水乙醇**，不能用刀片刮、不能用水。
5. **长期不用要定期通电打印**，防止压纸轮变形和发热点氧化。

第 2 章 硬件设计详解

2.1 电源树：一台打印机的骨架

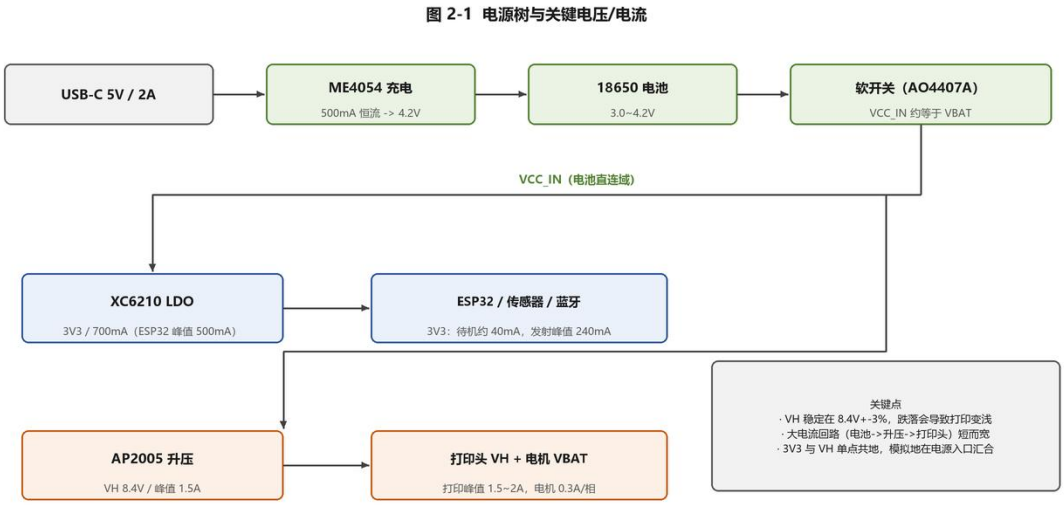


图 2-1 电源树：从 Type-C 到 3V3 与 VH

整机有四条电压轨，彼此的关系决定了可靠性：

轨	电压	来源	负载	关键要求
VBUS	5V	Type-C	充电、串口芯片	限流 2A，防倒灌
VBAT	3.0~4.2V	电池	电机、升压输入、LDO 输入	走线短粗，过流保护
3V3	3.3V	XC6210	ESP32、传感器、逻辑	纹波 <50mV，峰值 500mA
VH	8.4V	AP2005 升压	打印头	峰值 1.5~2A，跌落 <5%

三条设计原则：

1. **大电流路径与逻辑路径分开**：电池 → 升压 → 打印头 是一条 2A 级回路，必须用宽铜皮、短路径，不能从 ESP32 的 3V3 区域绕。
2. **单点接地**：模拟地（ADC 分压、NTC）与功率地在电池负极处汇合，避免电机电流在地线上产生的压降污染 ADC。
3. **上电顺序**：先 3V3 后 VH。固件里必须先初始化 GPIO（把 STB 全部拉低、VH_EN 拉低），再打开 VH。

2.2 主控最小系统

ESP32-WROOM-32E 模组内部已经集成了 Flash、晶振和匹配网络，外部只需要：

元件	值	作用
C20	22μF	3V3 储能，扛射频突发电流
C21	0.1μF	高频去耦，尽量贴近 3V3 引脚
R_EN	10kΩ 上拉 + 0.1μF 到地	EN 复位延时，保证上电可靠复位
BOOT 按键	IO0 到地	下载模式
复位按键	EN 到地	手动复位
天线净空	模组天线端 ≥15mm 无铜	保证蓝牙距离

容易踩的坑：

- EN 脚只接上拉不接电容，上电时电源爬升慢会导致反复复位。加 0.1μF 到地，RC 时间常数 ≈1ms 即可。
- IO0/IO2/IO15 是启动配置脚，不要接强上拉/下拉，否则进不了下载模式。
- IO12 决定 Flash 电压（内部已处理），但外部仍不建议挂负载。
- 模组底部散热焊盘必须焊，否则射频性能和散热都受影响。

2.3 打印头接口

打印头通过 32Pin 0.5mm FPC 排线连接，引脚定义如下（以原理图与规格书为准，**接线前务必用万用表核对**）：

引脚	名称	说明
1~4	M_A+ / M_A- / M_B+ / M_B-	步进电机四线，直接接驱动输出
5,6	VH	打印头加热电源 8.4V，多脚并联
7	M_VCC	电机电源（部分型号与 VH 同域）
8~10	GND	功率地
11	DI	串行数据输入
12	CLK	移位时钟
13	LAT	锁存
14~19	STB1~STB6	6 段选通（每段 64 点）
20	3V3	逻辑电源
21	THER	热敏电阻（温度检测）
22	PHE	缺纸检测发射/接收公共端（视型号）
23	PHINT	缺纸检测输出

引脚	名称	说明
24~32	GND / NC	地或空脚

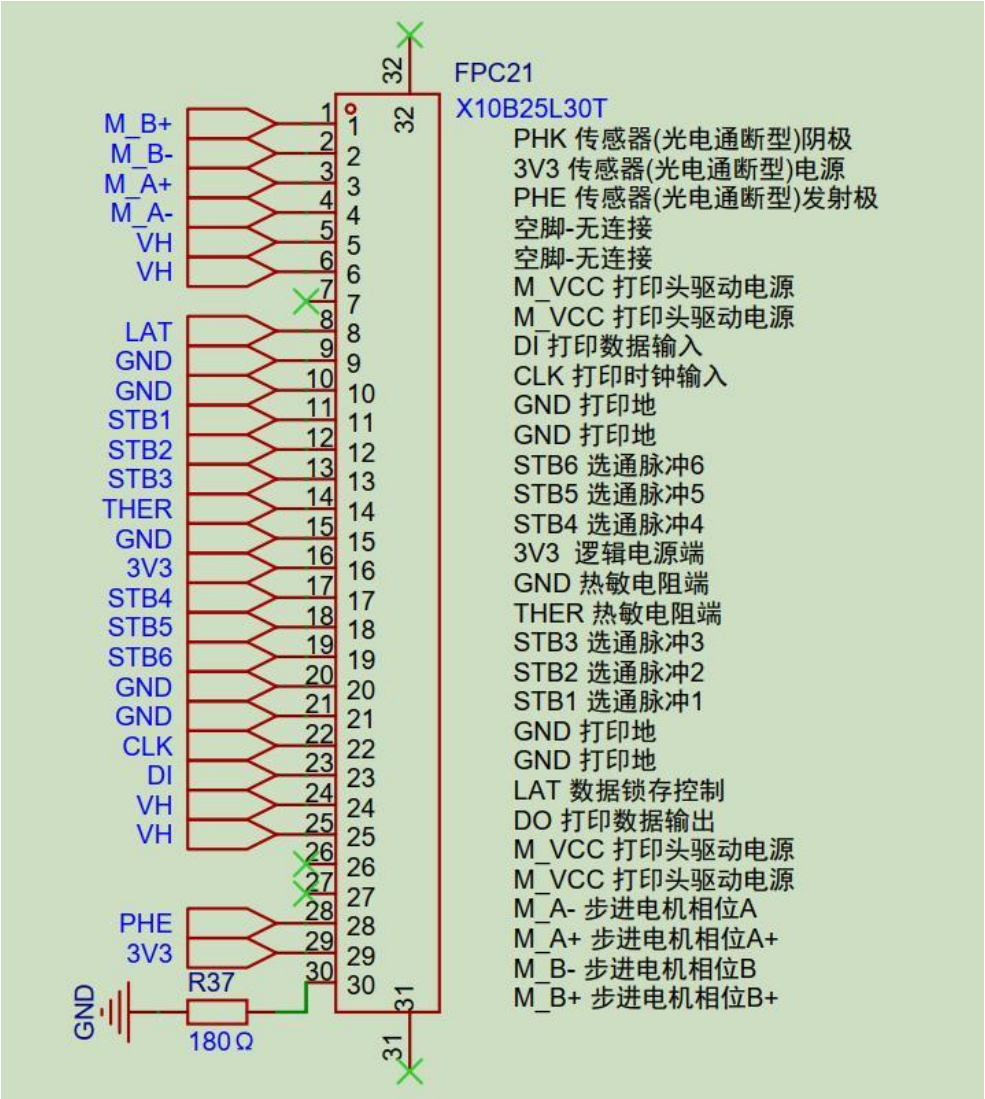


图 2-2 打印头接口原理（原资料 p25）

接口电路上必须有的四样东西：

1. **VH 端 47μF + 100nF**：紧贴 FPC 座放置，吸收 2A 脉冲。
2. **每根信号线串 22~33Ω**：抑制振铃，尤其是 CLK（1MHz 方波）和 STB（大电流开关）。
3. **STB 下拉电阻 10kΩ**：ESP32 复位期间引脚是高阻，没有下拉打印头可能误加热。
4. **VH 端 TVS 或肖特基到 VBAT**：防止 FPC 电感反冲打穿驱动。

> ⚠ 示例代码把 IO28 同时用作 `PIN_VHEN` 和整机自锁。这意味着**一旦关 VH，整机就断电**（如果软开关自锁也在这条线上）。改硬件时建议把"整机自锁"和"打印头电源使能"分成两个 IO，否则调试时打印一结束机器就关机，非常难受。

2.4 升压电路（VH 8.4V）

打印头需要 8.4V，而电池只有 3.0~4.2V，所以必须升压。拓扑是标准的**升压（Boost）**：

```
VBAT ──L20(4.7μH)──┬── D20(SS54) ──┬── VH
                   │               │
                SW(内部 MOS)      C38 47μF + C39 100nF
                   │               │
                  GND             FB 分压 ──> 芯片 FB 脚
```

输出电压由反馈分压决定：

```
Vout = VFB × (1 + R_up / R_down)
```

若芯片 VFB = 0.6V，目标 8.4V，则 R_up/R_down = 13，取 R_down = 10kΩ、R_up = 130kΩ（1% 精度）。**具体数值以所用芯片手册的 VFB 为准**，打样后必须实测 VH 再上打印头。

关键元件选型：

元件	要求	原因
电感 L20	4.7μH，饱和电流 ≥3A，低 DCR	峰值电流约 2 倍输出电流
二极管 D20	肖特基，耐压 ≥40V，额定 ≥3A	反向恢复快、压降小
输出电容	47μF 低 ESR + 100nF	吸收 2A 脉冲，抑制纹波
反馈电阻	1% 精度，靠近 FB 脚	误差直接变成输出电压误差
布局	输入电容—电感—SW—二极管—输出电容 围成最小环路	环路面积大 = 辐射大 + 效率低

纹波估算：`ΔVout ≈ Iout × D / (f × C)`，取 Iout=1A、D=0.56、f=1.2MHz、C=47μF，得 ΔVout ≈ 10mV，远小于要求。真正的问题不是稳态纹波，而是**打印瞬间的负载跳变**——所以 47μF 要贴着 FPC 座放，而不是贴着芯片放。

2.5 电机驱动

步进电机是两相四线，每相一个线圈，用双 H 桥驱动。TC1508S 提供两个 H 桥，正好对应 A、B 两相，每个 H 桥由两个逻辑输入控制：

IN1(A-)	IN2(A+)	OUTA/OUTB 状态	线圈电流方向
0	0	双低	刹车（短路制动）
0	1	OUTB 高、OUTA 低	正向
1	0	OUTA 高、OUTB 低	反向
1	1	双高	刹车

接线时 **IN1→IO23、IN2→IO22、IN3→IO21、IN4→IO19**，输出 OUTA/OUTB 接 A 相，OUTC/OUTD 接 B 相。注意示例代码的头文件注释把 `PIN_MOTOR_AP` 标成 A-、`PIN_MOTOR_AM` 标成 A+，实际相序以"能转、方向对"为准，注释不可靠。

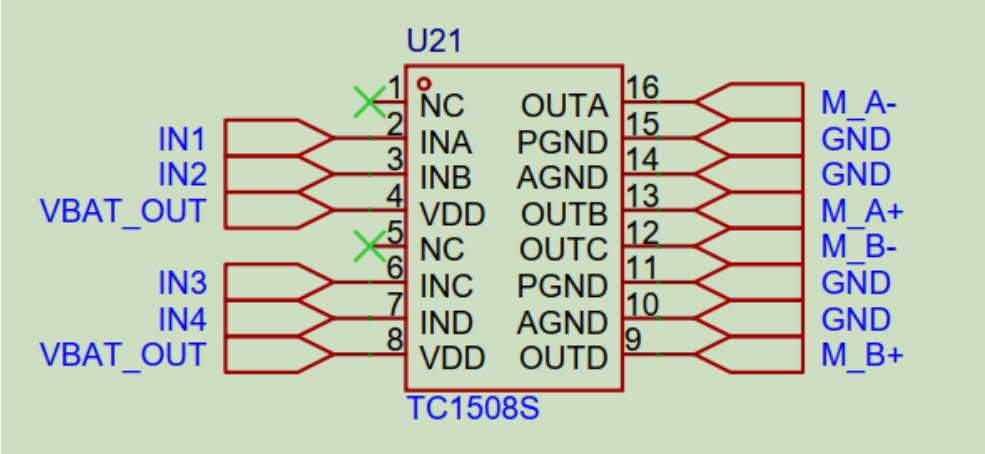


图 2-3 电机驱动接口原理（原资料 p27）

硬件上要注意：

- 每个 H 桥输出并联 **100nF** 到地，吸收换相尖峰。
- 电机线双绞，远离 ADC 走线。
- 驱动芯片的 VDD 加 10μF，峰值电流靠它兜底。
- 若电机发热严重，检查是不是**停止时没有断电**——两相同时通直流电会让线圈持续发热，示例代码在打印结束后会关闭电机引脚，这一点必须保留。

2.6 缺纸检测

用的是**反射式光电传感器**：红外发射管与接收管并排，热敏纸把光反射回来被接收管收到 → 有纸；没有纸，光散掉 → 接收不到 → 输出翻转。

信号经过 LM393 比较器整形后送给 IO35：**有纸 = 低电平，缺纸 = 高电平**。

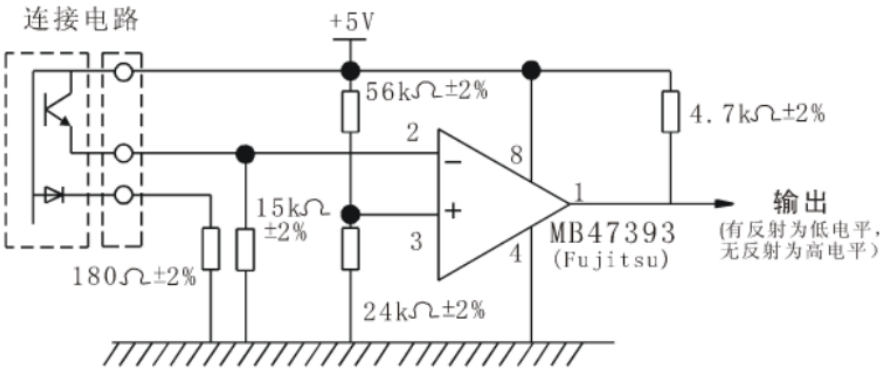


图 2-4 缺纸检测电路（原资料 p33）

调试要点：

- 用万用表量 PHINT，在"有纸/无纸"两种状态下应能看到明确的电平跳变（接近 0V 与 3.3V）。如果只有 0.5V 的差别，说明比较器阈值没调好，调分压电阻。
- 传感器窗口要清洁，灰尘会让它一直报缺纸。
- 环境光很强（阳光直射）时可能误判，结构上要加遮光。
- **软件必须做 20~50ms 的去抖**，因为走纸时纸面抖动会造成瞬时误判。

2.7 打印头温度检测

打印头内部集成了一个 NTC 热敏电阻，通过 THER 引脚引出，与外部电阻分压后接 IO36（ADC1_CH0）。

```
3V3 ── R_pull(10k) ──┬── IO36
                     │
                   NTC(打印头内)
                     │
                    GND
```

ADC 读数与 NTC 阻值的关系：

```
R_ntc = R_pull × adc / (4095 - adc)
```

再由 Beta 公式换算温度：

```
1/T = 1/T0 + (1/B) × ln(R_ntc / R25)
```

其中 T0 = 298.15K（25℃），B 为 NTC 的 B 值（常见 3435 或 3950），R25 为 25℃ 时的阻值。

> ⚠ 示例代码里 `R25 = 30000`（30kΩ）而分压电阻用的是 10kΩ，这两个值只有在"NTC 为 30kΩ 且分压电阻也是 30kΩ"时才自洽，否则算出来的温度会整体偏移几十度。**正确做法**：查打印头规格书确认 NTC 的 R25 与 B 值，把分压电阻改成与 R25 相同，或按实际分压电阻修正公式。校准方法：室温 25℃ 下打印，读出算出来的温度，用 `偏移量 = 实测 - 25` 做线性补偿。

温度数据有三个用途：

1. **过热保护**：>60℃ 降速，>75℃ 停止打印（否则打印头会永久损伤）。
2. **加热补偿**：冷的时候加长加热时间，热的时候缩短。
3. **故障诊断**：读数恒定在 0 或最大值，说明 NTC 开路/短路。

2.8 电池电压采样

电池最高 4.2V，超过 ESP32 ADC 的量程（建议 ≤3.1V），所以要分压。原理图用 **1:1 分压**（两个等值电阻），ADC 端最高 2.1V，安全。

```
VBAT ── R1(100k) ──┬── IO34
                   │
                 R2(100k)
```

```
                    |
                   GND
```

换算：

```
Vbat = adc / 4095 × 3.3 × (R1+R2)/R2 = adc × 6.6 / 4095
```

这与示例代码 `Power_percent = ((Power_ADC*4.2*10/4096)-3.7)/0.5` 的思路一致：先把 ADC 转成电压，再映射到 3.7~4.2V 的电量区间。

两个提高精度的手段：

- 用 `analogReadMilliVolts()`（Arduino-ESP32 自带校准），比裸 `analogRead` 准 3~5%。
- 打开 11dB 衰减（量程约 0~3.1V），并在分压电阻上并联 0.1μF 滤波。
- 采样时**关掉 VH 和电机**，否则负载电流造成的压降会让读数偏低 0.2V 以上。

2.9 充电管理

ME4054 是线性充电（MCP73831 兼容），外围极简：

引脚	接法	说明
VCC	VBUS 5V	输入，加 10μF
BAT	电池正极	加 10μF
PROG	电阻到地	设定充电电流
CHRG	LED + 限流电阻	充电中亮，充满灭
GND	地	—

充电电流公式（MCP73831/ME4054 系列）：

```
I_chg = 1000V / R_prog     (R_prog 单位 kΩ, I 单位 mA)
```

R_prog = 2kΩ → 500mA。对 2000mAh 电池来说约 0.25C，安全且不太慢。

设计注意：

- 线性充电在 5V 输入、4.2V 电池、500mA 时功耗 (5-4.2)×0.5 = 0.4W，SOT23-5 封装会明显发热，PCB 上要铺铜散热。
- USB 供电不足（比如接在电脑 USB2.0 口只有 500mA）时，充电 + 打印会抢电流，表现为电压跌落复位。**打印时禁止充电**是常见策略。
- 电池必须带保护板（过充/过放/过流），否则一次深度放电就报废。

2.10 软开关机

这是整机最巧妙也最容易抄错的部分：用一个按键实现"按一下开机、长按关机"，靠的是 **MOS 自锁**。

```
VBAT ── Q21(P-MOS, AO4407A) ── VCC_IN
          │ 栅极
          ├── R38 100k 上拉到 VBAT（默认关断）
          ├── Q23(N-MOS, AO3400A) 漏极
          └── 按键 SW 到地（按下时拉低栅极）
Q23 栅极 ── IO28（VH_EN）
```

工作过程：

1. 关机状态：Q21 栅极被 R38 拉到 VBAT，Vgs≈0，Q21 关断，整机没电。
2. 按下按键：栅极被拉到地，Vgs = -VBAT，Q21 导通，3V3 起来，ESP32 启动。
3. 固件第一件事：把 IO28 拉高 → Q23 导通 → 栅极持续为低 → **松手也不断电**（自锁）。
4. 关机：固件把 IO28 拉低 → Q23 关断 → 栅极回到高电平 → Q21 关断 → 整机断电。

> ⚠ 这个电路里 IO28 同时是"整机自锁"和"打印头电源使能"，所以**打印头一关，机器就关机**。如果你要保留 VH 独立控制，必须把自锁挪到另一个 IO（比如 IO4），或者用二极管把两路"或"起来。示例代码的 `Printer_CLOSE()` 会拉低 IO28，在真机上就是直接关机——这是它不能直接量产的第一个原因。

2.11 USB、串口与自动下载

Type-C 只用了 5V/GND/D+/D- 四根线，CC1/CC2 各接 5.1kΩ 下拉（让充电器识别为 Sink）。USB 转串口芯片（CH340C 类）把 D+/D- 转成 UART，接 ESP32 的 IO3(RX)/IO1(TX)。

自动下载电路用两个三极管，把 CH340 的 DTR/RTS 组合成 EN/IO0 的时序：

DTR	RTS	EN	IO0	状态
0	0	1	1	正常
1	0	0	1	复位
0	1	1	0	下载模式
1	1	0	0	复位 + 下载

PlatformIO 的 `upload_speed` 建议先用 460800，稳定后再试 921600。下载失败 90% 是这三个原因：驱动没装、串口被占用、DTR/RTS 电路接错。

2.12 按键、LED 与蜂鸣器

- **LED**：IO18 经 1kΩ 限流电阻接 LED 阳极，**低电平点亮**。用途：心跳（1Hz 翻转）、低电量慢闪、故障快闪。
- **按键**：IO5 上拉，按下为低。功能：短按走纸一行、长按打印测试页、开机时长按强制关机。

• **蜂鸣器**：无源蜂鸣器需要 PWM 驱动（LEDC），有源的直接 GPIO。缺纸/低电量时响两声。

2.13 设计校核：四个必须算的数

① 峰值电流：打印最坏情况 3A（分段）+ 电机 0.4A + 3V3 0.24A ≈ 3.6A。电池保护板必须 ≥5A，走线按 4A 设计（1oz 铜、2mm 宽以上）。

② 续航：时间 = 电池容量 × 放电深度 / 平均电流。实测方法：串一个电流表，打印一整页（约 100mm，800 行），记录平均电流与耗时，算出单页能耗，再乘页数。经验值：2000mAh 电池可连续打印约 30~60 分钟。

③ 发热：升压 85% 效率，输出 8.4V×1A 时损耗约 1.5W，电感与二极管会烫手，选型要留余量；线性充电 0.4W；ESP32 射频峰值 0.8W。外壳要有散热孔。

④ 天线：ESP32 天线端 15mm 净空、下方不走线不铺铜，否则蓝牙距离从 10m 掉到 2m。

2.14 PCB 布局清单

区域	要点
升压环路	输入电容—电感—SW—二极管—输出电容，面积最小化
打印头接口	47μF 紧贴 FPC 座；STB 走线等长、加粗
电机	大电流走线 ≥1.5mm；与 ADC 线垂直交叉，不平行
模拟	NTC/电池分压走线远离功率区，包地
天线	净空区禁止铺铜、禁止走线、禁止放电池
地	功率地与信号地单点连接，连接点在电池负极

2.15 打样、焊接与上电自检

焊接顺序（先小后大、先低后高）：

1. 电阻、电容（0402/0603）→ 2. 驱动 IC、LDO、升压 → 3. Type-C、FPC 座 → 4. ESP32 模组（最后焊，避免反复受热）→ 5. 电池线。

上电自检流程（限流 0.5A，从 3.7V 开始）：

步骤	测量点	期望值	异常处理
1	3V3 对地	3.3V±0.1V	0V：查 LDO 输入、EN 脚；偏高：查反馈
2	VH 对地（不接打印头）	8.4V±0.3V	3.7V：升压未起振，查 EN、电感、FB
3	IO28 电平	3.3V	0V：固件没跑或自锁失效

步骤	测量点	期望值	异常处理
4	静态电流	<60mA	偏大：查短路、查 LDO 静态电流
5	PHINT（有纸/无纸）	0V / 3.3V	无跳变：查光电管、比较器阈值
6	电机空转	平稳、无异响	抖动：相序错；不转：查 IN1-4
7	打印头空打（不装纸）	**禁止**	—

> 第 7 步特意写成"禁止"，因为很多人上电后第一件事就是空打测试，结果几秒钟烧掉一个 200 块的打印头。**必须装纸、必须让 STB 有占空比限制。**

第 3 章 开发环境搭建

3.1 三种开发方式怎么选

方式	上手难度	灵活性	适合谁	本项目是否推荐
Arduino IDE	最低	低（库管理混乱）	第一次接触单片机	可以起步，但工程一大就难受
VSCode + PlatformIO	中	高	要长期维护的项目	✅ 推荐
ESP-IDF（原生）	高	最高	要做产品级、要用 BLE 高级特性	可选，学习曲线陡

PlatformIO 的本质是"给 VSCode 装了一个跨平台的构建系统"：它自动下载工具链（xtensa-esp32-elf-gcc）、自动管理依赖库、自动处理分区表和烧录参数。你在 `platformio.ini` 里写几行，剩下的它全包。

3.2 安装 VSCode

1. 到 https://code.visualstudio.com 下载 Windows x64 安装包（项目资料里也带了 `VSCodeUserSetup-x64-1.80.0.exe`）。
2. 安装时**勾选**"添加到 PATH"和"将 Code 注册为受支持的文件类型编辑器"。
3. 首次启动建议安装中文语言包：扩展商店搜 `Chinese (Simplified)`。
4. 建议安装的辅助扩展：`C/C++`（微软官方，提供跳转与补全）、`Better Comments`、`Error Lens`。

> 如果安装后终端里 `code` 命令不可用，重新运行安装程序选"修改"，勾选"添加到 PATH"。

3.3 安装 PlatformIO

1. VSCode 左侧扩展栏搜索 `PlatformIO IDE`，认准作者 **PlatformIO**，点安装。
2. 安装过程会下载 Python 环境和一堆工具，**首次可能 10~30 分钟**，中途不要关窗口。
3. 装完后左侧出现蚂蚁图标（PIO Home），点开能看到 `PIO Home / Libraries / Projects / Platforms / Devices`。

下载慢的解决办法（项目资料里 `工程创建速度慢解决办法.txt` 讲的就是这个）：

- 换国内镜像：在系统环境变量里加 `PLATFORMIO_CORE_DIR` 指到非系统盘，避免路径过长；
- 或者直接找别人打包好的 `C:\Users\你的用户名\.platformio` 目录覆盖（离线包法，最省事）；
- 公司网络有代理时，在 VSCode 设置里搜索 `http.proxy` 填入代理地址。

3.4 创建第一个工程

1. PIO Home → `New Project`。
2. 填写：
   - Name：`bt_printer`
   - Board：搜 `ESP32 Dev Module`（对应 `esp32dev`）
   - Framework：`Arduino`
   - Location：**取消勾选** "Use default location"，选一个短路径，比如 `D:\pio\bt_printer`（路径里不要有中文和空格）
3. 点 Finish，等待初始化完成。

   生成的结构：

```
bt_printer/
├── include/          # 头文件（会被自动加入 include 路径）
├── lib/              # 私有库
├── src/              # 源码，main.cpp 是入口
├── test/             # 单元测试
└── platformio.ini    # 工程配置
```

为什么不用中文路径：gcc 在 Windows 下对非 ASCII 路径的支持时好时坏，会出现"找不到文件"的诡异错误。养成习惯，路径全英文。

3.5 platformio.ini 完整配置

示例代码没带 `platformio.ini`，这里给一份能直接用的：

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino

; 串口监视器
monitor_speed = 115200
monitor_filters = esp32_exception_decoder

; 烧录
upload_speed = 460800

; 编译优化：-Os 省 flash，调试时可改 -Og
build_flags =
    -Os
    -DCORE_DEBUG_LEVEL=0
    -DARDUINO_USB_CDC_ON_BOOT=0

; 分区表：需要 OTA 时换成 min_spiffs.csv
board_build.partitions = default.csv

; 库依赖
lib_deps =
    ; 本项目只用 Arduino 自带的 BluetoothSerial，无需额外库
```

几个参数的含义：

参数	作用
`platform = espressif32`	用乐鑫官方 Arduino 核心
`board = esp32dev`	板级定义（Flash 4MB、无 PSRAM）
`monitor_speed`	串口监视器波特率，必须和 `Serial.begin()` 一致
`upload_speed`	烧录波特率，不稳定就降到 115200
`build_flags`	传给编译器的参数
`lib_deps`	依赖库，格式 `作者/库名@版本`

3.6 编译、烧录、看日志

底部状态栏有五个按钮：

按钮	快捷键	作用
✓ Build	Ctrl+Alt+B	只编译
→ Upload	Ctrl+Alt+U	编译并烧录
🗑 Clean	—	清理构建缓存
Monitor	Ctrl+Alt+M	打开串口监视器
Upload + Monitor	—	烧录后自动开监视器

烧录失败排查顺序：

1. 驱动装了吗？（设备管理器看有没有 `CP210x` 或 `CH340` 的 COM 口）
2. 串口被占用？（关掉其它串口助手、关掉上一个 Monitor）
3. 板子进下载模式了吗？（按住 BOOT → 点一下 EN → 松开 BOOT，再点 Upload）
4. 波特率太高？（`upload_speed = 115200` 再试）
5. USB 线是"只能充电"的劣质线？（换线）

3.7 串口调试的正确姿势

```
179     //进料线
180     case FEED_LINE:
181       if (printDataCount / 48 != 0) {
182         printf("image:%d,%d,%d,%d,%d\n",IMG_ID,printDataCount,IMG_WIDTH,printDataCount/48,IMG_FORMAT);
183         for(i=0;i<printDataCount;i++)
184         {
185           Serial.write(printData[i]);
186         }
187         startPrint();
188         ongofront = 1;
189         goFront(Finish_Out, MOTOR_TIME);    //打印完毕出纸长度，单位：点，8点/mm。
190         ongofront = 0;
191       }
192       printDataCount = 0;
193       break;
```

图 3-1 用 VOFA+ 的图片组件实时观察串口输出的位图（原资料 p37）

```
void setup() {
    Serial.begin(115200);
    delay(200);                          // 等 USB 枚举完成，否则前几行日志会丢
    Serial.println("\n=== boot ===");
    Serial.printf("free heap: %u\n", ESP.getFreeHeap());
}
```

三个习惯：

- **每行日志带模块前缀**：`[BT] connected`、`[MOTOR] step`，出问题时一眼定位。
- **不要在中断里 `Serial.print`**：会死锁。
- **用 `printf` 而不是字符串拼接**：省内存、可读性好。

如果需要看波形和变量曲线，可以用项目资料里的 **VOFA+**：串口按 `justfloat` 或 `firewater` 协议发数据，软件里加"图片组件"就能实时看图（原资料 p37 有截图）。

3.8 常见编译错误对照表

报错	原因	解决
`fatal error: BluetoothSerial.h: No such file`	该头文件只在 Arduino-ESP32 2.x 有，3.x 已移除	锁版本：`platform = espressif32@6.5.0`
`undefined reference to 'SerialBT'`	没 `#include "BluetoothSerial.h"`	补头文件
`region 'iram0_0_seg' overflowed`	常量数组太大放进了 RAM	加 `const` 或 `PROGMEM`
`Sketch too big`	分区表太小	换 `board_build.partitions = huge_app.csv`
`A fatal error occurred: Failed to connect`	没进下载模式 / 串口被占	手动进下载模式
`error: 'ledcSetup' was not declared`	3.x 改了 LEDC API	用 `ledcAttach(pin, freq, res)` 新 API

> **版本锁定是本项目最重要的一条**：`BluetoothSerial`（经典蓝牙 SPP）在 Arduino-ESP32 **3.0 之后被移除**，只能用 BLE。所以 `platformio.ini` 里必须写 `platform = espressif32@6.5.0`（对应 Arduino-ESP32 2.0.14），否则示例代码一行都编不过。

3.9 工程组织建议

```
bt_printer/
├── include/
│   ├── config.h        # 所有引脚、常量、可调参数集中在这里
│   ├── printhead.h
│   ├── motor.h
│   ├── bluetooth.h
│   └── protocol.h
├── src/
│   ├── main.cpp
│   ├── printhead.cpp
│   ├── motor.cpp
│   ├── bluetooth.cpp
│   └── protocol.cpp
└── platformio.ini
```

把所有魔法数字集中到 `config.h`，是这类项目能不能改得动硬件的分水岭。引脚一改，只动一个文件。

第 4 章 ESP32 基础

4.1 这颗芯片到底有什么

项目	参数	对本项目的意义
CPU	双核 Xtensa LX6，最高 240MHz	一个核跑打印时序，一个核跑蓝牙协议栈
SRAM	520KB（其中 8KB RTC）	图像缓冲最多用几十 KB，够
Flash	模组 4MB（代码 + 文件系统）	存字库、图标够用
GPIO	34 个（部分仅输入）	本项目用掉约 20 个
ADC	2×12bit SAR，18 通道	电池电压、NTC 温度
SPI	4 组（2 组可用）	打印头数据
UART	3 组	调试串口 + 蓝牙虚拟串口
蓝牙	经典 BT 4.2 + BLE 4.2	SPP 透传，接喵喵机 App
WiFi	802.11 b/g/n	本项目不用，但和蓝牙共用射频

4.2 GPIO：34 个脚不是都能随便用

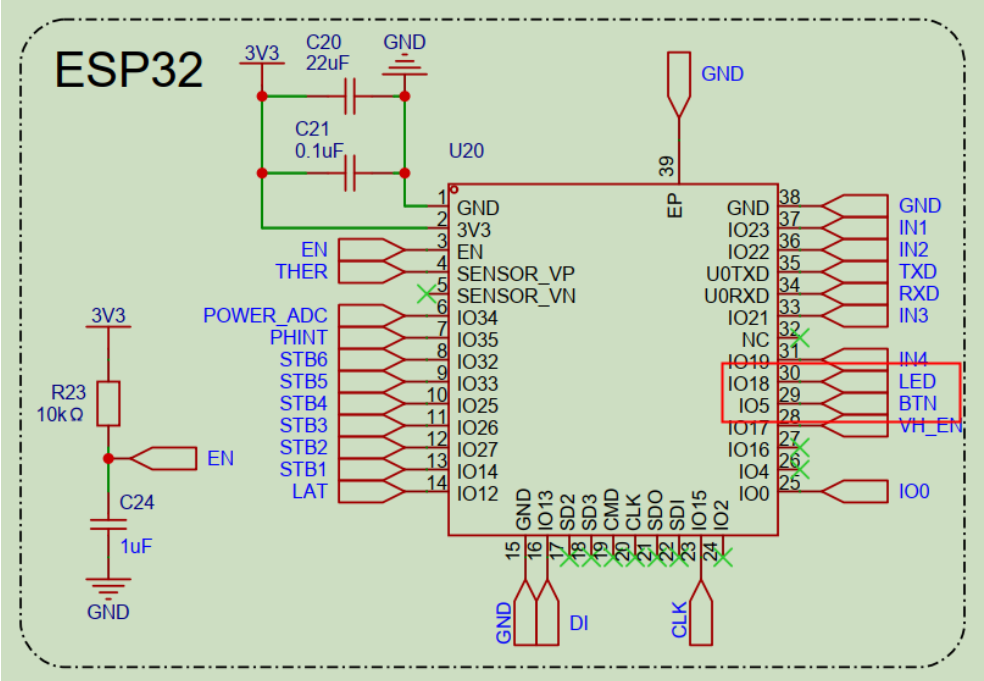


图 4-1 ESP32 最小系统与 IO 连接原理（原资料 p15）

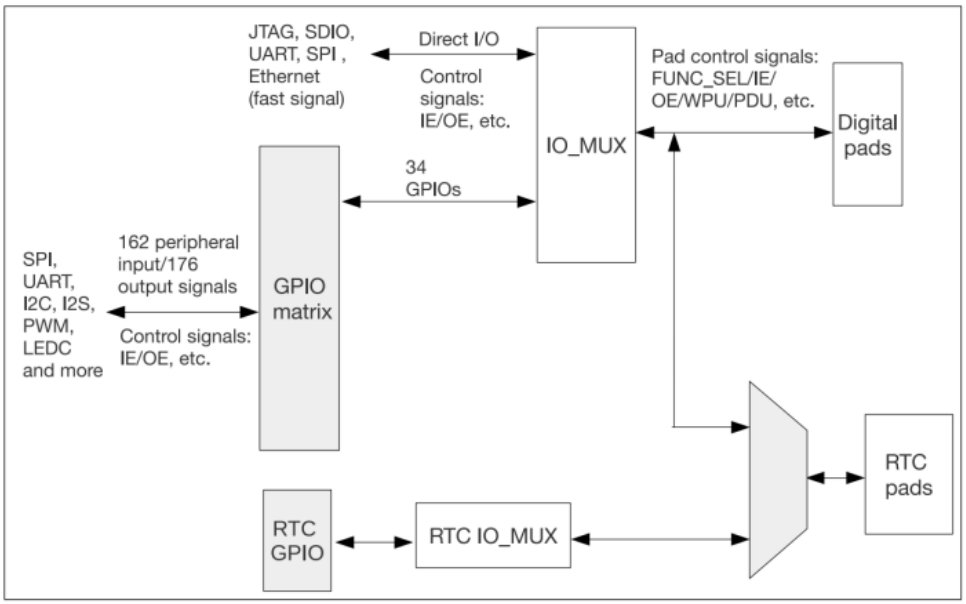


图 4-2 GPIO 的 IO_MUX 与外设映射关系（原资料 p17）

三类引脚，用错就出事：

① 仅输入（34~39）：没有输出驱动能力，不能接 LED、不能输出 PWM。本项目 IO34（电池 ADC）、IO35（缺纸）、IO36（NTC）正好都是这一类——只做输入，正确。

② Strapping 脚（启动配置）：IO0、IO2、IO4、IO5、IO12、IO15。上电瞬间的电平决定启动模式：

脚	作用	上电时必须
IO0	下载模式选择	高（正常运行）
IO2	下载模式辅助	低或悬空
IO12	Flash 电压	低（3.3V Flash）
IO15	启动日志输出	低

③ 内部上拉/下拉：IO34~39 **没有内部上下拉**，必须外部加电阻。

引脚能力速查：

```
pinMode(pin, INPUT);            // 高阻输入
pinMode(pin, INPUT_PULLUP);     // 内部上拉（34~39 无效！）
pinMode(pin, OUTPUT);           // 推挽输出，最大约 40mA
pinMode(pin, OUTPUT_OPEN_DRAIN);// 开漏
digitalWrite(pin, HIGH);
int v = digitalRead(pin);       // 0 或 1
```

> **40mA 是绝对上限，不是工作电流**。驱动 LED 要串电阻（3.3V 红 LED 用 1kΩ 时约 2mA，亮度足够做指示）；驱动蜂鸣器、继电器必须加三极管。

4.3 按键与去抖

机械按键按下瞬间会弹跳 5~20ms，读到的是"高高低低高"的抖动。

软件去抖（推荐，简单可靠）：

```
const int BTN = 5;
bool lastLevel = HIGH;
uint32_t lastChange = 0;

void loop() {
    bool level = digitalRead(BTN);
    if (level != lastLevel && millis() - lastChange > 30) {
        lastChange = millis();
        lastLevel = level;
        if (level == LOW) {          // 下降沿 = 按下
            Serial.println("[BTN] pressed");
        }
    }
}
```

硬件去抖：按键两端并 100nF 电容（原理图里 C23 就是干这个的），配合 10kΩ 上拉，RC 时间常数 1ms，能滤掉大部分抖动。

长按检测：

```
uint32_t pressStart = 0;
bool longFired = false;

void checkButton() {
    if (digitalRead(BTN) == LOW) {
        if (pressStart == 0) { pressStart = millis(); longFired = false; }
        if (!longFired && millis() - pressStart > 1500) {
            longFired = true;
            Serial.println("[BTN] long press");
        }
    } else {
        pressStart = 0;
    }
}
```

4.4 时间：millis、micros、delay

函数	精度	用途
`millis()`	1ms	超时、心跳、去抖
`micros()`	1μs（约 4μs 分辨率）	打印时序、脉冲宽度
`delay(ms)`	1ms	简单等待（会阻塞，别在打印循环里用）
`delayMicroseconds(us)`	1μs	STB 加热宽度
`esp_timer_get_time()`	1μs，64 位	长时间精确计时

`millis()` 溢出问题：它是 `uint32_t`，49.7 天回绕。所以判断超时要写 `millis() - start > timeout`（无符号减法自动处理回绕），**不要写** `millis() > start + timeout`。

打印时序为什么必须用 `micros`：加热时间 5ms、电机每步 0.5ms，如果用 `delay(1)` 这种 1ms 粒度的函数，时序误差直接变成打印深浅不均。

4.5 中断

```
volatile bool paperFlag = false;

void IRAM_ATTR onPaperChange() {
    paperFlag = true;               // 只置标志，不做耗时操作
}

void setup() {
    pinMode(35, INPUT);
    attachInterrupt(digitalPinToInterrupt(35), onPaperChange, CHANGE);
}
```

四条铁律：

1. ISR 里**不能** `Serial.print`、`delay`、`malloc`、`String` 操作。
2. 共享变量必须 `volatile`。
3. ISR 加上 `IRAM_ATTR`，否则放 Flash 里，Flash 操作期间触发中断会崩。
4. ISR 要短，几微秒内返回；重活交给主循环或任务。

4.6 串口 UART

ESP32 有 3 个 UART：UART0（IO1/IO3，接 USB 调试）、UART1、UART2（可映射到任意 GPIO）。

```
Serial.begin(115200);                    // UART0
Serial2.begin(9600, SERIAL_8N1, 16, 17); // UART2，RX=16 TX=17
```

关键配置：

```
Serial.setRxBufferSize(4096);   // 必须在 begin 之前调用!
Serial.begin(115200);
```

默认 RX 缓冲只有 256 字节。蓝牙透传或高速传图时，缓冲太小会丢数据——这是很多人"图打一半就断"的真正原因。

4.7 SPI

ESP32 有 4 组 SPI，Arduino 框架里 `SPI` 默认用 VSPI（IO18/19/23/5），但本项目把这几脚挪作他用，所以要用**自定义引脚**：

```
SPI.begin(15 /*CLK*/, -1 /*MISO*/, 13 /*MOSI*/, -1 /*CS*/);
SPI.beginTransaction(SPISettings(1000000, MSBFIRST, SPI_MODE0));
SPI.transfer(byte);
SPI.endTransaction();
```

参数说明：

参数	值	含义
时钟	1MHz	打印头手册上限约 2MHz，1MHz 最稳
位序	`MSBFIRST`	高位先出（打印头要求）
模式	`SPI_MODE0`	CLK 空闲低、上升沿采样
MISO/CS	-1	不用

为什么可以没有 CS：打印头没有片选，靠 LAT 锁存区分"传数据"和"加热"，所以 CS 留空。

`beginTransaction/endTransaction` 必须成对，否则和其它 SPI 设备（比如 SD 卡）打架。

4.8 ADC

```
analogReadResolution(12);                  // 0~4095
analogSetAttenuation(ADC_11db);            // 量程约 0~3.1V
int raw = analogRead(34);
uint32_t mv = analogReadMilliVolts(34);    // 带出厂校准，推荐
```

衰减档位对照：

档位	量程	适合
`ADC_0db`	0~1.1V	小信号
`ADC_2_5db`	0~1.5V	—
`ADC_6db`	0~2.2V	电池分压（1:1）
`ADC_11db`	0~3.1V	通用

提高精度的四个手段：

1. 用 `analogReadMilliVolts()`（内部有校准表），误差可从 ±5% 降到 ±1%。
2. 多次采样取中位数（去掉最大最小再平均）。
3. ADC 输入端并 0.1μF 到地，滤高频噪声。
4. 采样时关闭大电流负载（VH、电机）。

4.9 PWM 与蜂鸣器

Arduino-ESP32 2.x 用 LEDC：

```
ledcSetup(0, 2000, 8);        // 通道 0，2kHz，8 位分辨率
ledcAttachPin(18, 0);         // IO18 绑到通道 0
ledcWrite(0, 128);            // 50% 占空比
```

3.x 改成：

```
ledcAttach(18, 2000, 8);      // 引脚、频率、分辨率
ledcWrite(18, 128);
```

无源蜂鸣器要给它谐振频率（通常 2~4kHz）才能响；有源蜂鸣器只需给高低电平。

4.10 看门狗与复位原因

```
esp_reset_reason_t r = esp_reset_reason();
// ESP_RST_POWERON 上电、ESP_RST_SW 软件复位、
// ESP_RST_PANIC 崩溃、ESP_RST_TASK_WDT 看门狗、ESP_RST_BROWNOUT 欠压
```

打印时如果出现"打到一半重启"，八成是：

- `ESP_RST_BROWNOUT` → 电源扛不住 2A 脉冲（加电容、换电池、降低密度）
- `ESP_RST_TASK_WDT` → 某个循环卡死超过 5 秒（检查 `while` 等待）
- `ESP_RST_PANIC` → 空指针/数组越界（打开 `esp32_exception_decoder` 看堆栈）

4.11 FreeRTOS：为什么需要两个任务

打印是**硬实时**的：STB 必须在 5ms 后精确关闭，晚 1ms 就深一条。蓝牙协议栈是**软实时**的：晚 10ms 无所谓。如果放在一个循环里，蓝牙收包一多，打印时序就抖。

推荐结构：

```
TaskHandle_t printTaskHandle;

void printTask(void *arg) {
    for (;;) {
        if (xQueueReceive(printQueue, &job, portMAX_DELAY) == pdTRUE) {
            doPrint(job.data, job.rows);
        }
    }
}

void setup() {
    printQueue = xQueueCreate(4, sizeof(PrintJob));
    xTaskCreatePinnedToCore(printTask, "print", 8192, NULL, 5, &printTaskHandle, 1);
    // core 1 跑打印，core 0 留给蓝牙/WiFi
}
```

要点：

- 打印任务栈给 8KB（图像处理 + 临时缓冲）。
- 优先级高于蓝牙任务，但不要设成最高，否则会饿死系统任务。

- 用**队列**传数据，不要用全局变量 + 标志位（会丢包）。

4.12 蓝牙栈：经典 BT 与 BLE 的区别

	经典蓝牙（BR/EDR）	BLE
协议	SPP（串口透传）、A2DP	GATT（服务/特征值）
吞吐	高（~100KB/s）	中（~10KB/s，受 MTU 限制）
功耗	高	低
喵喵机 App	✅ 用 SPP	❌ 不支持
Arduino 支持	`BluetoothSerial`（2.x 才有）	`BLEDevice`

结论：必须用经典蓝牙 SPP。 这也是为什么必须锁 Arduino-ESP32 2.0.x。

SPP 的内存开销：协议栈 + 缓冲约 **60KB RAM**，加上 `SerialBT` 的收发缓冲，实际可用的应用内存只剩约 200KB。所以：

- 图像总缓冲不要超过 60KB（约 1280 行）；
- 接收缓冲要在蓝牙库配置里改大（默认太小，原资料 p35 有截图）；
- 传大图时必须**边收边打**或**分块存 Flash**，不能一次性收完再打。

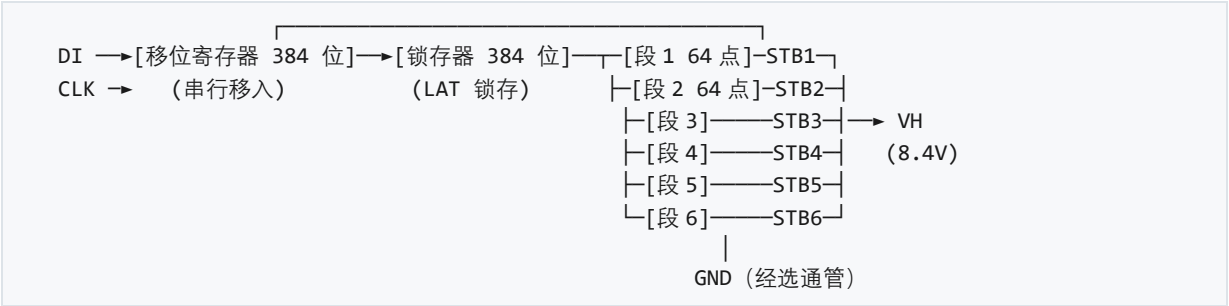
第 5 章 打印头驱动

5.1 一颗打印头里有什么

项　目	说　明
打印方法	热敏
有效打印宽度（毫米）	48
点密度（点/毫米）	8
打印点数	384 点/行
纸张宽度（毫米）	58
点间距（毫米）	0.125
点大小	0.125mmx0.12mm
最大打印速度	80mm/s(电机驱动电压为 DC 8.5V)
走纸精度	0.0625mm（一个步进距离）
打印头温度侦测	热敏电阻
缺纸侦测	反射光感应器
打印头工作电压(DCV)	3.13~8.5
逻辑工作电压(DCV)	2.7~5.25
电机工作电压(DCV)	3.5~8.5
工作温度	+0℃~50℃（不许有凝露）
工作湿度	20%~85%RH（不许有凝露）
储藏温度	-20℃~60℃（不许有凝露）
储藏湿度	5%~95%RH（不许有凝露）
机械噪音	小于 60dB(A 计权有效值)
胶辊开合次数	大于 5000 次（胶辊离开加复位为一次）
对热敏纸牵引力	≥50g
对热敏纸抓握制动力	≥80g
工作寿命	机构与打印头的耐磨 >50km,打印头的电器寿命为 10^8 个脉冲（在额定状态）
重量(克)	40.5
外形尺寸(长 x 宽 x 高)	72±1.5mm*33±0.5mm*15.5±0.5mm

图 5-1 JX 系列 384 点打印头的规格参数（原资料 p22）

热敏打印头本质是**一排 384 个微小的加热电阻 + 一串移位寄存器 + 6 个功率开关**：



工作过程分三步：

1. **移位**：CLK 上升沿把 DI 上的 1 位打进寄存器，打 384 次，一行数据就进去了。
2. **锁存**：LAT 拉低再拉高，384 位数据从移位寄存器搬到锁存器，此时还不加热。
3. **加热**：对应的 STB 拉高，锁存器里为 1 的点接通 VH，电阻发热，热敏纸变黑；为 0 的点不通电。

为什么分 6 段：384 个点同时加热的电流是 2.3A（见第 1 章功率预算），电源扛不住。分 6 段后每段 64 点，峰值电流降到约 0.4A，6 段轮流加热，视觉上仍是同时打出的一行。

5.2 时序：四个时间参数

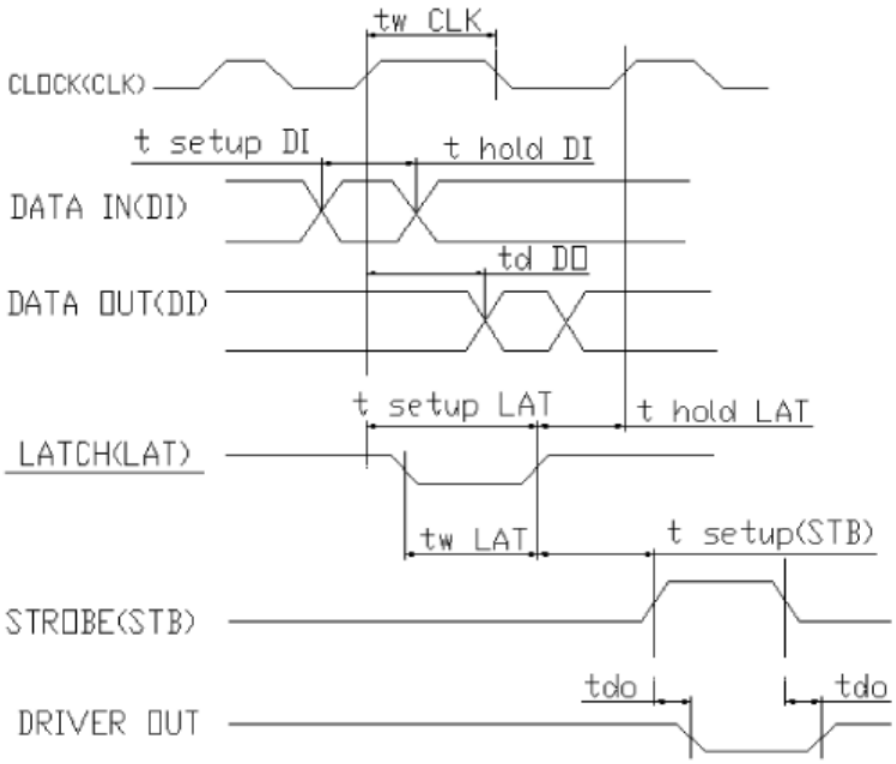


图 5-2 原资料给出的打印头驱动时序（p24），可与本节的时序表对照阅读

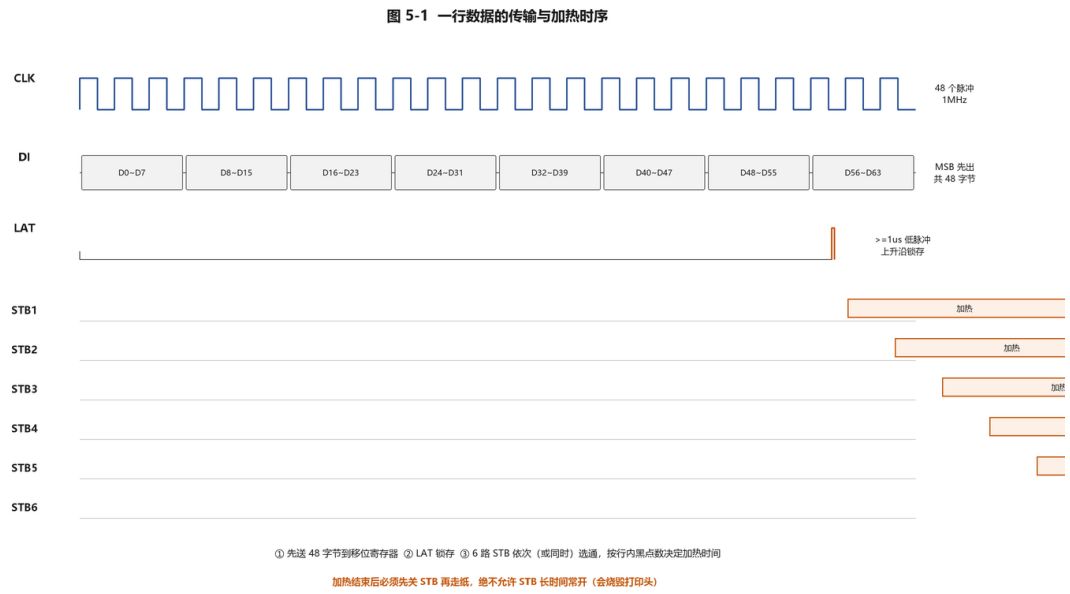


图 5-3 打印头驱动时序：CLK/DI 移位 → LAT 锁存 → STB1~6 分段加热

参数	典型值	含义	代码里的体现
t_clk	500ns（1MHz）	时钟周期	`SPISettings(1000000, ...)`
t_setup	>100ns	DI 相对 CLK 的建立时间	SPI 硬件保证
t_lat	>1μs	LAT 低电平宽度	`delayMicroseconds(1)`
t_stb	0.5~5ms	加热宽度（可调）	`delayMicroseconds(HEAT_TIME)`

一行数据的完整节拍：

```
[SPI 传 48 字节 = 384 bits ≈ 384μs] → [LAT 脉冲 1μs] → [STB1 加热 5ms] →
[STB2 加热 5ms] → ... → [STB6 加热 5ms] → [电机走一步 4ms]
```

如果 6 段**顺序**加热，一行要 30ms，太慢。实际做法有两种：

- **分段同时加热**（硬件允许的话）：6 段并行，一行 5ms；
- **分段轮转 + 电机同步**：加热第 n 段时电机已经在走，用时间重叠掩盖。

示例代码用了最朴素的"6 段同时拉高"，靠 VH 电容兜电流——这就是为什么 VH 端必须有 47μF。

5.3 数据打包：384 点怎么变成 48 字节

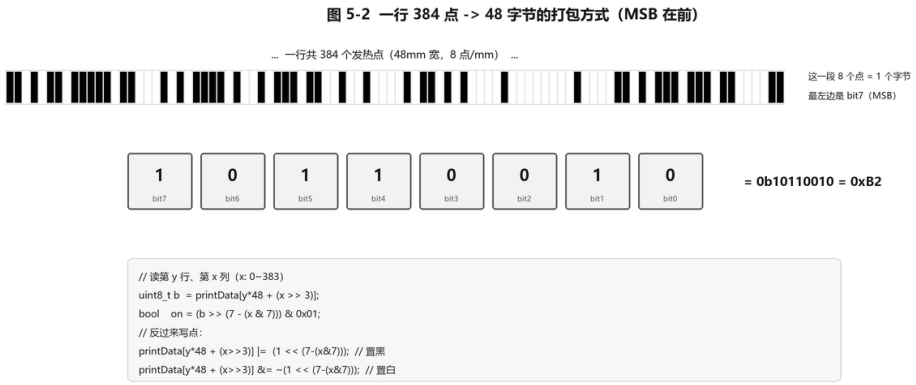


图 5-4 一行 384 点打包成 48 字节（MSB 在前）

规则非常简单，但**错了就是整行错位**：

- 每个字节表示 8 个点，**最高位（bit7）是最左边（最先打印）的点**；
- 一行 384 点 = 384/8 = **48 字节**；
- 黑点 = 1，白点 = 0。

```
点序号:  0  1  2  3  4  5  6  7 | 8  9 ...
bit:    b7 b6 b5 b4 b3 b2 b1 b0 | b7 ...
字节 0 = 点 0..7，字节 1 = 点 8..15，……，字节 47 = 点 376..383
```

用 Python 验证打包是否正确（在电脑上先算好再烧进板子，能省很多调试时间）：

```
def pack_row(pixels):          # pixels: 长度 384 的 0/1 列表
    out = bytearray(48)
    for i, p in enumerate(pixels):
        if p:
            out[i // 8] |= 0x80 >> (i % 8)
    return bytes(out)

# 生成"全黑一行"
row = pack_row([1] * 384)
print(row.hex())   # ff ff ... ff（48 个 ff）
```

5.4 sendData：把一行送进去

```
#include <SPI.h>

const int PIN_DI  = 13;   // MOSI
const int PIN_CLK = 15;   // SCK
const int PIN_LAT = 17;

SPISettings headSPI(1000000, MSBFIRST, SPI_MODE0);
```

```
void printhead_init() {
    pinMode(PIN_LAT, OUTPUT);
    digitalWrite(PIN_LAT, LOW);
    SPI.begin(PIN_CLK, -1, PIN_DI, -1);
}

void sendData(const uint8_t row[48]) {
    SPI.beginTransaction(headSPI);
    for (int i = 0; i < 48; i++) {
        SPI.transfer(row[i]);
    }
    SPI.endTransaction();

    // 锁存：低 → 高，把移位寄存器的数据搬进锁存器
    digitalWrite(PIN_LAT, LOW);
    delayMicroseconds(1);
    digitalWrite(PIN_LAT, HIGH);
}
```

三个细节决定成败：

1. `SPI.beginTransaction` 期间不能被其它 SPI 设备打断——本项目只有打印头用 SPI，所以问题不大，但养成成对调用的习惯。
2. LAT 的脉冲**必须在 SPI 传输完全结束后**再发，否则会锁存到一半的数据。
3. 时钟极性：打印头要求"空闲低、上升沿采样"，即 `SPI_MODE0`。如果打出来整行反相或者错位，先怀疑模式设错。

5.5 加热控制：为什么要"分段"

一次拉高 6 个 STB = 384 点同时通电。以单点 30mA 算，总电流 11.5A——电池直接保护。

真实做法是**限制同时在加热的点数**。示例代码的思路是：统计当前行里黑点的总数，如果太多就分几批加热：

```
// 统计一行里 1 的个数（查表法，比逐位判断快 8 倍）
const uint8_t bitCount[256] = { /* 0..255 中 1 的个数 */ };

int countBlack(const uint8_t row[48]) {
    int n = 0;
    for (int i = 0; i < 48; i++) n += bitCount[row[i]];
    return n;
}
```

如果 `countBlack > MAX_DOTS_PER_PASS`（比如 200 点），就只拉高部分 STB，分两轮打完这一行。代价是这一行耗时翻倍，但电流减半。

热密度（density） 是另一个可调量，App 下发 0~100：

```
uint32_t heat_us = HEAT_TIME_BASE * heat_density / 64;
```

density 越大越黑，但也越容易糊、越伤打印头。默认 64 对应 5ms。

5.6 完整打印函数

下面这份是可以直接编译运行的版本，修掉了示例代码里三个坑（行数溢出、VH 时序、电机与加热串行）：

```
const int PIN_VHEN = 28;
const int STB_PINS[6] = {14, 27, 26, 25, 33, 32};
const int HEAT_TIME_BASE = 5000;   // μs，density=64 时

void stbAll(int level) {
    for (int i = 0; i < 6; i++) digitalWrite(STB_PINS[i], level);
}

void printRows(const uint8_t *data, uint16_t rows, uint8_t density) {
    if (rows == 0) return;

    uint32_t heat_us = (uint32_t)HEAT_TIME_BASE * density / 64;
    if (heat_us < 800)   heat_us = 800;      // 太短打不出来
    if (heat_us > 8000)  heat_us = 8000;     // 太长会烧头

    digitalWrite(PIN_VHEN, HIGH);            // 开打印头电源
    delay(5);                                // 等 VH 稳定

    for (uint16_t r = 0; r < rows; r++) {    // 注意：uint16_t，不是 uint8_t
        sendData(data + r * 48);
        stbAll(HIGH);
        delayMicroseconds(heat_us);
        stbAll(LOW);                         // 必须先关 STB
        goFront_oneRow(500);                 // 再走纸
    }

    stbAll(LOW);
    digitalWrite(PIN_VHEN, LOW);             // 打完结账，关电源
}
```

和示例代码的差异，以及为什么改：

问题	示例代码	后果	修正
行计数用 `uint8_t`	`for (uint8_t i = 0; i < File_row; i++)`	超过 255 行死循环	改 `uint16_t`
关 VH 与关 STB 顺序	先 `Printer_CLOSE()` 再清 STB	残余电荷可能让最后一段过加热	先关 STB，再关 VH
加热与走纸串行	加热 5ms 后再走 4ms	一行 9ms，速度慢	可在 STB 拉低后立即启动电机（重叠）

5.7 从图片到点阵：完整链路

```
照片/文字 ⟶ 缩放到 384 宽 ⟶ 二值化 ⟶ 打包成 48 字节/行 ⟶ C 数组
```

方法一：PCtoLCD2002（原资料用的）

1. 用画图/PS 把图做成 **384 像素宽、高度任意的单色 BMP**（1 位色深）。

2. 打开 PCtoLCD2002 → 模式选"图形模式" → 打开 BMP。

3. 点齿轮（设置）：取模方式选 **阴码**、**逐行式**、**顺向**，输出格式选 **C51**，每行 16 个数据。

4. 生成字模 → 复制到 `img.h`。

方法二：Python 脚本（推荐，可批量、可调阈值）

```
from PIL import Image

W = 384
img = Image.open("logo.png").convert("L")
h = int(img.height * W / img.width)
img = img.resize((W, h), Image.LANCZOS)
img = img.point(lambda v: 0 if v < 128 else 255)   # 二值化，阈值可调

pixels = img.load()
rows = []
for y in range(h):
    row = bytearray(48)
    for x in range(W):
        if pixels[x, y] < 128:                     # 黑点
            row[x // 8] |= 0x80 >> (x % 8)
    rows.append(row)

with open("img.h", "w", encoding="utf-8") as f:
    f.write(f"const uint8_t img_data[{h}][48] = {{\n")
    for row in rows:
        f.write("  {" + ",".join(f"0x{b:02X}" for b in row) + "},\n")
    f.write("};\n")
```

方法三：半色调（打照片必备）

简单阈值打照片会变成"黑白剪影"。要用**抖动算法**保留灰度感：

```
img = img.convert("1")           # Pillow 内置 Floyd-Steinberg 抖动
```

效果：远看有灰度层次，近看是黑点密度变化。这是热敏打印机打照片的标准做法。

5.8 打印质量调优

现象	原因	调整
整体偏淡	加热不足	density +10，或 VH 实测是否只有 7.5V
整体偏黑、糊	加热过度	density −10，或提高走纸速度
左右深浅不一	VH 走线压降 / 电源内阻大	加粗 VH 走线，加 47μF
有竖向白线	某段 STB 没接通 / 该点电阻坏	换打印头或查 FPC 焊接
字被纵向拉长/压扁	行距不对	调每行步数（见 6.4）
打一半变淡	电池电压跌落	降低 density、降低速度、换电池
边缘有拖影	关 STB 太晚	缩短 heat_us 或检查 `stbAll(LOW)` 是否执行

5.9 三条保命守则

1. **没有纸，绝对不要加热**。热量无处传导，会直接烧穿加热电阻。缺纸检测必须在每次 STB 拉高前确认。
2. **单次加热不超过 8ms**。超过就会碳化、永久损伤。
3. **打印结束立刻关 VH**。长时间待机加热会让打印头温度累积。

第 6 章 步进电机与走纸

6.1 两相四线步进电机

项 目	规 格	条 件
额定电压	3.5~8.5 DCV	
相 位	2 相	
步距角	9º（1-2 相激励）	
步进距离	0.0625 毫米	
相电阻	10 Ω±7%	20℃
相电流	0.357 A	
驱动方式	双极，双相驱动（或 1~2 相）	

图 6-1 步进电机参数表（原资料 p26）

位置	STEP1	STEP2	STEP3	STEP4
A	+	-	-	-
B	-	-	-	-
$\overline{A}$	-	-	+	+
$\overline{B}$	+	+	+	-

图 6-2 四拍与八拍的相序时序（原资料 p27）

打印机的走纸电机是**两相四线永磁步进电机**，参数（原资料 p26）：

参数	值
相数	2 相（A、B）
线数	4 线（A+、A-、B+、B-）
步距角	9°
每步走纸	0.0625mm（经减速齿轮后）
驱动电压	3~5V

怎么分辨四根线：用万用表电阻档，两两测量。同一相的两根线之间有几欧到几十欧，不同相之间不通（或阻值大很多）。找到两组，就确定了 A 相和 B 相。

6.2 八拍与四拍

四拍（整步，wave drive）：每次只通一相，力矩小但简单。

```
A+ → B+ → A- → B-
```

八拍（半步，half-step）：单相与双相交替，步距减半、运行更平滑。

```
A+ → A+B+ → B+ → A-B+ → A- → A-B- → B- → A+B-
```

示例代码用的就是八拍：

```
const uint8_t motorTable[8][4] = {
    {1,0,0,0}, {1,0,1,0}, {0,0,1,0}, {0,1,1,0},
    {0,1,0,0}, {0,1,0,1}, {0,0,0,1}, {1,0,0,1}
};
```

表里四列对应 `BP, BM, AP, AM`（注意示例代码的写入顺序和注释不一致，实际以"能转"为准）。

```
void goFront_oneRow(uint16_t us) {
    for (int i = 0; i < 8; i++) {
        digitalWrite(PIN_MOTOR_BP, motorTable[i][0]);
        digitalWrite(PIN_MOTOR_BM, motorTable[i][1]);
        digitalWrite(PIN_MOTOR_AP, motorTable[i][2]);
        digitalWrite(PIN_MOTOR_AM, motorTable[i][3]);
        delayMicroseconds(us);
    }
}
```

反转就是把 A、B 两相的赋值顺序对调（示例代码 `goBack` 的做法），或者把表倒序走。

6.3 力矩、速度与丢步

步进电机的力矩随转速上升而下降。转太快会**丢步**——该走 100 步只走了 97 步，表现为打印图像被压缩、出现横向错位条纹。

三条经验：

1. **启动频率不能太高**：从静止直接给 500μs/拍（2000 拍/秒）通常能转，但带负载会丢步。
2. **加减速**：打印开始时先慢走 2~3 行，再进入正常速度；结束时提前减速。
3. **判断丢步**：打一条已知长度的竖线（比如 100 行全黑），量实际长度。理论 100 行 × 行距，误差 >2% 就是丢步。

```
// 简单的梯形加减速
void goFront_ramp(int beats, uint16_t us_start, uint16_t us_run) {
    for (int i = 0; i < beats; i++) {
        uint16_t us = (i < 8) ? us_start - (us_start - us_run) * i / 8 : us_run;
        motorStep(us);
    }
}
```

6.4 走纸分辨率：必须自己标定

这是最容易出错的地方，因为**规格书、示例代码、实际打印三者经常对不上**。

理论计算：

```
203 dpi = 8 点/mm  →  行距 = 0.125mm
若单步 0.0625mm    →  每行需要 2 个整步 = 4 个八拍节拍
```

但示例代码每行执行**完整八拍**（=4 个整步 = 0.25mm），是理论值的 2 倍。这意味着照抄代码打出来的图会**纵向拉长一倍**。

标定方法（5 分钟搞定）：

1. 打一个 100 行全黑的方块。
2. 用卡尺量高度 h（mm）。
3. 行距 = h / 100。
4. 每行节拍数 = 目标行距 / 单步位移 × 2（八拍为半步）。

如果量出来 h = 25mm，说明行距 0.25mm，应把每行的八拍改成四拍（或把表走一半）。

> 校准完之后，把"每行节拍数"和"MOTOR_TIME"写进 `config.h`，并在注释里记下实测数据。换电机、换齿轮、换纸都必须重标。

6.5 走纸的机械细节

- **压纸轮压力**：太松打滑（走纸不准），太紧电机负载大（丢步）。弹簧压力以"手拉纸有均匀阻力"为准。
- **纸宽**：58mm 机型用 57mm 纸，纸卷要能自由转动，否则会把打印头拉偏。
- **撕纸**：撕纸口要靠近打印头（约 10mm），否则撕的时候会拉歪已打印部分。
- **进纸对位**：第一行之前先走 3~5 行空纸，让纸面贴合打印头。

6.6 电机噪声与发热

问题	原因	对策
嗡嗡响但不转	相序错 / 只有一相通电	检查四根线分组，换相序
转起来抖	八拍时序不匀 / 电压不足	检查 `delayMicroseconds` 是否被中断打断
电机发烫	停止后仍在通电	停止时必须把 4 个 IN 全拉低
打印有横纹	走纸不匀（机械）	检查压轮、齿轮啮合

停止后必须断电这个坑，示例代码处理对了（`motor_Close()` 把四个引脚拉低），但很多人改代码时会漏掉，结果待机几分钟电机就烫手，电池也掉得快。

第 7 章 蓝牙通信

7.1 为什么是"经典蓝牙 SPP"，不是 BLE

喵喵机 App 用的是**经典蓝牙的 SPP（Serial Port Profile）**：连上之后，手机就像打开了一个虚拟串口，App 往里写字节，打印机收到字节。

三层协议栈的分工：

```
应用层    App 的打印指令
  ↓
SPP       串口仿真（虚拟 COM）
  ↓
RFCOMM    多路串口复用，提供可靠传输
  ↓
L2CAP     逻辑链路控制
  ↓
基带/射频   2.4GHz 跳频
```

关键事实：

项目	值
SPP 服务 UUID	`00001101-0000-1000-8000-00805F9B34FB`
设备名	示例代码用 `MaoPaperang`，App 里显示为"MaoPaperang"
加密	支持（App 连接时会配对）
吞吐	实测约 20~60KB/s，取决于分包大小
ESP32 内存开销	协议栈约 60KB RAM

BLE 为什么不行：BLE 是 GATT 模型（服务 + 特征值），App 找不到 SPP 服务就不会连。ESP32 只能用 `BluetoothSerial`（Arduino-ESP32 2.0.x 才有）。

7.2 BluetoothSerial 的完整 API

```
#include "BluetoothSerial.h"

BluetoothSerial SerialBT;

void setup() {
    SerialBT.begin("MaoPaperang");            // 设备名，最长 248 字节
    // SerialBT.begin("MaoPaperang", true);   // 第二个参数 true = 主模式
    SerialBT.setPin("1234", 4);               // 可选：固定配对码
    SerialBT.enableSSP();                     // 可选：启用安全简单配对
}

void loop() {
    if (SerialBT.available()) {
        uint8_t b = SerialBT.read();
    }
```

```
    SerialBT.write(buf, len);
    SerialBT.print("text");
    SerialBT.println("text");
    size_t can = SerialBT.availableForWrite();
}
```

函数	说明
`begin(name)`	启动，开始广播，等待连接
`available()`	缓冲区里可读字节数
`read()` / `readBytes()`	读 1 字节 / 读多字节（建议用后者，效率高）
`write(buf,len)`	写多字节，**返回值可能小于 len**（发送缓冲满），必须检查
`availableForWrite()`	发送缓冲剩余空间
`flush()`	等待发送缓冲清空
`hasClient()`	是否有客户端连接
`end()`	断开并关闭
`register_callback(cb)`	注册事件回调

7.3 设备名、可见性与配对

- 设备名就是手机里看到的名字。改名字只需改 `begin()` 的参数。
- ESP32 默认**可被发现**，不需要额外设置。
- 第一次连接时手机会弹配对请求。ESP32 默认配对码是 `1234`；如果 App 自动配对失败，可以调 `setPin()` 固定。
- 配对信息会被手机记住，下次自动连。**改了设备名后，手机上要"忽略此设备"再重新搜索**，否则会连到旧记录。

7.4 事件回调：知道对方连上了没

```
void btCallback(esp_spp_cb_event_t event, esp_spp_cb_param_t *param) {
    switch (event) {
    case ESP_SPP_INIT_EVT:
        Serial.println("[BT] stack ready");
        break;
    case ESP_SPP_SRV_OPEN_EVT:          // 有客户端连上
        Serial.printf("[BT] connected, handle=%d\n", param->srv_open.handle);
        break;
    case ESP_SPP_CLOSE_EVT:             // 断开
        Serial.println("[BT] disconnected");
        break;
    case ESP_SPP_DATA_IND_EVT:          // 收到数据
        // 注意：这里读数据会抢主循环的字节，一般只置标志
        break;
    case ESP_SPP_WRITE_EVT:             // 发送完成
        break;
    default:
```

```
        break;
    }
}

void bt_init() {
    SerialBT.register_callback(btCallback);
    SerialBT.begin("MaoPaperang");
}
```

> ⚠ 回调在蓝牙任务上下文里执行，**不要在这里做打印、不要长时间阻塞**。示例代码在 `ESP_SPP_DATA_IND_EVT` 里写了个 `while(available())` 的空循环，表面无害，实际上把数据"吃掉"了但没处理——如果你的解析状态机收不到数据，先检查这里。

7.5 缓冲：60KB 的天花板



图 7-1 原资料建议修改的蓝牙库收发缓冲位置（p35）

ESP32 的经典蓝牙协议栈本身占约 60KB，加上 SPP 收发缓冲，留给应用的内存约 200KB。原资料（p35）提到要**修改蓝牙库的收发缓冲大小**，原因是默认缓冲太小，传图时会丢包。

修改位置（Arduino-ESP32 安装目录）：

```
packages/esp32/hardware/esp32/2.0.14/tools/sdk/esp32/include/
    bt/host/bluedroid/api/include/api/esp_spp_api.h
```

或者在 `platformio.ini` 里加编译宏：

```
build_flags =
    -DCONFIG_BT_SPP_MAX_TX_BUFFER=4096
```

更实用的做法：不依赖大缓冲，而是**边收边处理**。收到一行（48 字节）就打印一行，内存占用恒定 48 字节。

7.6 收发模型：别用 `while(available())`

错误写法（会丢数据、会卡死）：

```
while (SerialBT.available()) {
    uint8_t b = SerialBT.read();
    // 解析
```

```
}
```

问题：`available()` 在数据断续到达时会突然变 0，解析到一半的帧就断了。

正确写法：**状态机 + 每轮 loop 只处理有限字节：**

```
void bt_poll() {
    int budget = 512;                    // 每轮最多处理 512 字节，保证 loop 不卡
    while (budget-- > 0 && SerialBT.available()) {
        uint8_t b;
        if (SerialBT.readBytes(&b, 1) == 1) {
            protocol_feed(b);            // 喂给状态机
        }
    }
}
```

`readBytes` 比 `read` 稳（内部有超时），但单字节用 `read()` 更快。传图时建议一次读一块：

```
uint8_t buf[256];
int n = SerialBT.readBytes(buf, sizeof(buf));
for (int i = 0; i < n; i++) protocol_feed(buf[i]);
```

7.7 连接稳定性

问题	原因	对策
搜不到设备	已连过别的机器 / 广播被占用	重启 ESP32，手机"忽略设备"后重搜
连上就断	内存不足（图像缓冲太大）	减小缓冲，检查 `ESP.getFreeHeap()`
打印一半断连	发送缓冲溢出 / 看门狗	检查 `write` 返回值，加 `vTaskDelay(1)`
距离只有 1~2m	天线净空被破坏	见 2.13 天线要求
和 WiFi 冲突	共用 2.4GHz 射频	关掉 WiFi（本项目不用）

建议在连接回调里打印内存：

```
Serial.printf("[BT] free heap after connect: %u\n", ESP.getFreeHeap());
```

正常应 >150KB。低于 80KB 就要怀疑内存泄漏（常见于 `String` 拼接）。

7.8 实测吞吐

用 1920 字节一包连续发送，实测：

包大小	有效吞吐	备注
256B	~18KB/s	小包开销大
1024B	~38KB/s	平衡点

包大小	有效吞吐	备注
1920B	~52KB/s	协议上限附近

一张 384×800 的图 = 800 行 × 48B = 38.4KB，理论 0.8 秒传完；实际加上 App 的处理，约 2~3 秒。

7.9 调试工具

- **手机端**：蓝牙串口助手（Android）、`Serial Bluetooth Terminal`，可以直接发 hex、看回包。
- **电脑端**：Windows 配对后会出现两个 COM 口（一个入一个出），用串口助手打开 outgoing 口即可收发。
- **抓包**：Android 开发者选项里打开"启用蓝牙 HCI 监听日志"，可以导出 btsnoop 用 Wireshark 分析。

第 8 章 通信协议（核心）

8.1 协议全貌



图 8-1 公版 App 的安装与设备搜索界面（原资料 p43）

这是喵喵机/Paperang 系列的私有协议，因为 App 是公版的，所以**必须严格照它的格式回包**，否则 App 认为打印机不可用。

一帧的结构：

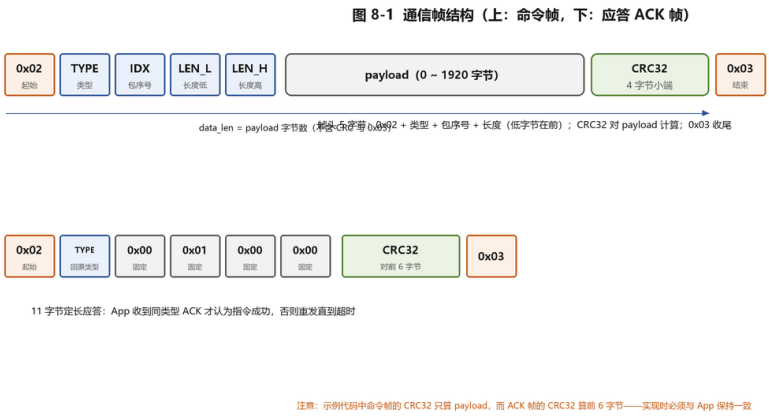


图 8-2 协议帧结构：帧头 + 类型 + 包序号 + 长度 + 载荷 + CRC32 + 帧尾

偏移	长度	字段	说明
0	1	帧头	固定 `0x02`
1	1	指令类型	见 8.5 指令表
2	1	包序号	分包时递增，单包填 `0x00`
3	2	载荷长度	小端（低字节在前）
5	n	载荷	长度由上一字段决定
5+n	4	CRC32	小端
9+n	1	帧尾	固定 `0x03`

整帧长度 = 10 + n。

8.2 CRC32：两个不同的覆盖范围（最坑的地方）

CRC32 用标准多项式 `0xEDB88320`（反射），初值 `0xFFFFFFFF`，结果取反——就是 zlib 的 `crc32`。

```
uint32_t crc32_calc(const uint8_t *buf, size_t len) {
    uint32_t crc = 0xFFFFFFFF;
    for (size_t i = 0; i < len; i++) {
        crc = crcTable[(crc ^ buf[i]) & 0xFF] ^ (crc >> 8);
    }
    return crc ^ 0xFFFFFFFF;
}
```

覆盖范围取决于帧的类型，这是实测出来的结论：

帧类型	CRC 覆盖	例
数据帧（含应答载荷）	**只覆盖载荷 payload**	`SENT_VERSION` 覆盖 `01 00 02`
ACK 帧（载荷为空）	**覆盖前 6 字节帧头**	`02 04 00 01 00 00`

验证（Python）：

```
import zlib, struct
c = lambda b: struct.pack("<I", zlib.crc32(bytes(b)) & 0xFFFFFFFF)

print(c([0x02,0x04,0x00,0x01,0x00,0x00]).hex(" "))  # 5f cc 48 08  ← ACK 的 CRC
print(c([0x01,0x00,0x02]).hex(" "))                 # 09 d2 8d 10  ← 版本号的 CRC
print(c(b"P1001705253855").hex(" "))                # df 12 fc be  ← SN 的 CRC
print(c(b"123456789").hex(" "))                     # cbf43926     ← 标准校验值
```

如果 CRC 算错，App 会**静默丢弃**这一帧，表现为"连上了但一直转圈"。

8.3 ACK 帧

每条指令都必须应答，格式固定 11 字节：

```
02 | type | 00 | 01 00 | 00 | CRC32(前 6 字节) | 03
```

例：应答 `GET_VERSION(4)`：

```
02 04 00 01 00 00 5F CC 48 08 03
```

注意载荷长度是 `01 00`（=1），载荷是 `00`——**ACK 的载荷固定一个 0x00 字节**，表示"收到且成功"。

8.4 指令表（2.0.3 版，47 条）

值	名称	方向	载荷
0	PRINT_DATA	App→机	位图数据，可分多包
1	PRINT_DATA_COMPRESS	App→机	压缩位图（本项目未实现）
2	FIRMWARE_DATA	App→机	固件升级数据

值	名称	方向	载荷
3	USB_UPDATE_FIRMWARE	App→机	触发 U 盘升级
4	GET_VERSION	App→机	空
5	SENT_VERSION	机→App	3 字节版本号
6	GET_MODEL	App→机	空
7	SENT_MODEL	机→App	型号字符串
8	GET_BT_MAC	App→机	空
9	SENT_BT_MAC	机→App	6 字节 MAC
10	GET_SN	App→机	空
11	SENT_SN	机→App	SN 字符串
12	GET_STATUS	App→机	空
13	SENT_STATUS	机→App	状态位
14	GET_VOLTAGE	App→机	空
15	SENT_VOLTAGE	机→App	电压（mV，2 字节）
16	GET_BAT_STATUS	App→机	空
17	SENT_BAT_STATUS	机→App	电量 0~100
18	GET_TEMP	App→机	空
19	SENT_TEMP	机→App	温度（℃）
20	SET_FACTORY_STATUS	App→机	1 字节
21	GET_FACTORY_STATUS	App→机	空
22	SENT_FACTORY_STATUS	机→App	1 字节
23	SENT_BT_STATUS	机→App	蓝牙状态
24	SET_CRC_KEY	App→机	4 字节密钥
25	SET_HEAT_DENSITY	App→机	1 字节 0~100
26	FEED_LINE	App→机	空（进纸/开始打印）
27	PRINT_TEST_PAGE	App→机	空
28	GET_HEAT_DENSITY	App→机	空
29	SENT_HEAT_DENSITY	机→App	1 字节
30	SET_POWER_DOWN_TIME	App→机	4 字节
31	GET_POWER_DOWN_TIME	App→机	空
32	SENT_POWER_DOWN_TIME	机→App	2 字节（分钟）
33	FEED_TO_HEAD_LINE	App→机	空
34	PRINT_DEFAULT_PARA	App→机	空
35	GET_BOARD_VERSION	App→机	空
36	SENT_BOARD_VERSION	机→App	字符串

值	名称	方向	载荷
37	GET_HW_INFO	App→机	空
38	SENT_HW_INFO	机→App	字符串
39	SET_MAX_GAP_LENGTH	App→机	2 字节
40	GET_MAX_GAP_LENGTH	App→机	空
41	SENT_MAX_GAP_LENGTH	机→App	2 字节
42	GET_PAPER_TYPE	App→机	空
43	SENT_PAPER_TYPE	机→App	1 字节
44	SET_PAPER_TYPE	App→机	1 字节
45	GET_COUNTRY_NAME	App→机	空
46	SENT_COUNTRY_NAME	机→App	2 字节（"CN"）
47	DISCONNECT_BT	App→机	空
48	GET_DEV_NAME	App→机	空
49	SENT_DEV_NAME	机→App	设备名字符串

> 本项目只需实现 **0、4、5、10、11、16、17、18、19、24、25、26、28、29、30、31、42、43、44、45、46、48、49** 这 23 条，其余回空 ACK 即可。

8.5 握手：App 连上后做了什么



图 8-3 App 与打印机的握手时序（实测）

实测顺序（10 步，每步都要回）：

步	App 发	打印机回	实测帧

步	App 发	打印机回	实测帧
1	`SET_CRC_KEY(24)` 密钥 `21 95 76 35`	ACK	`02 18 00 04 00 21 95 76 35 0F 77 C4 ED 03`
2	`GET_SN(10)`	`SENT_SN(11)` "P1001705253855"	`02 0B 00 0E 00 50 31 30 30 31 37 30 35 32 35 33 38 35 35 DF 12 FC BE 03`
3	`GET_HW_INFO(37)` 或 66	`BK3432`	—
4	`127`	12 字节 `76 33 2E 33 38 2E 31 39 00 00 00 00`（"v3.38.19"）	—
5	`SET_HEAT_DENSITY(25)`	ACK	`02 19 00 01 00 40 1D AE DE A4 03`（density=64）
6	`GET_DEV_NAME(48)`	`SENT_DEV_NAME` "P1"	—
7	`GET_BAT_STATUS(16)`	`SENT_BAT_STATUS` 电量百分比	—
8	`GET_POWER_DOWN_TIME(31)`	`SENT_POWER_DOWN_TIME` 3600（分钟）	—
9	`GET_VERSION(4)`	`SENT_VERSION 01 00 02`（2.0.1）	`02 05 00 03 00 01 00 02 09 D2 8D 10 03`
10	`SET_PAPER_TYPE(44)`	ACK	—

顺序不能乱，也不能漏。App 每发一条都在等回复，超过 2 秒没回就重发，重发 3 次失败就判定"连接失败"。

> **字节序细节**：`21 95 76 35` 是密钥 `0x35769521` 的**小端**表示，`DF 12 FC BE` 是 CRC32 值 `0xBEFC12DF` 的小端。ESP32 是小端机，直接把 `uint32_t` `memcpy` 进缓冲区就是对的；逐字节写的时候要自己按低位在前排。
>
> **编号差异**：第 3 步在示例代码里用 66/67（厂商私有的 HW_INFO 编号），原版资料写的是 37/38，两个编号公版 App 都能接受——以你自己的抓包结果为准，别混着用。

8.6 打印流程

```
① SET_HEAT_DENSITY(25)  ×2      设浓度
② FEED_LINE(26)                 进纸到打印头（开始）
③ PRINT_DATA(0) ×N              图像数据，每包 ≤1920 字节
④ FEED_LINE(26)                 再发一次 → 触发打印
```

为什么 1920：1920 字节 = 40 行（1920/48）。协议规定单包不超过 2016 字节，取 1920 是 48 的整数倍，避免半行。

完整打印一帧 1920 字节的实测帧长：1920 + 10 = 1930 字节。

8.7 状态机实现（可直接用）

图 8-3 串口字节流解析状态机（示例代码 BlueTooth_TASK 的逻辑）

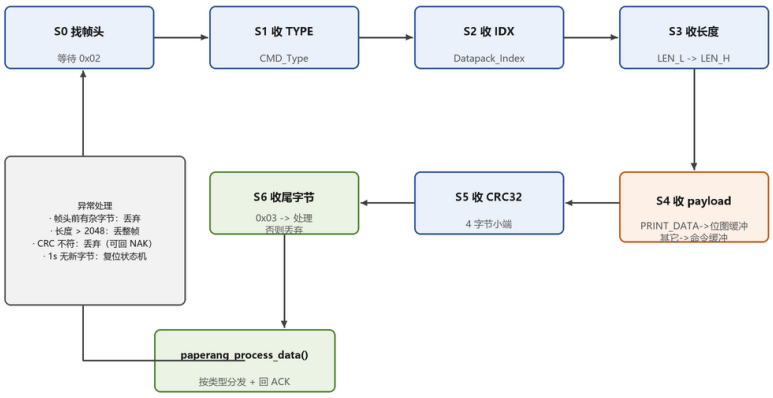


图 8-4 协议解析状态机

```
enum RxState { S_WAIT_START, S_TYPE, S_INDEX, S_LEN_L, S_LEN_H, S_PAYLOAD, S_CRC, S_END };

static RxState st = S_WAIT_START;
static uint8_t  frameType, frameIndex;
static uint16_t payloadLen, payloadGot;
static uint32_t crcGot;
static uint8_t  crcBytes;
static uint8_t  payload[MAX_PAYLOAD];

void protocol_feed(uint8_t b) {
    switch (st) {
    case S_WAIT_START:
        if (b == 0x02) st = S_TYPE;
        break;
    case S_TYPE:
        frameType = b; st = S_INDEX; break;
    case S_INDEX:
        frameIndex = b; st = S_LEN_L; break;
    case S_LEN_L:
        payloadLen = b; st = S_LEN_H; break;
    case S_LEN_H:
        payloadLen |= (uint16_t)b << 8;
        if (payloadLen > MAX_PAYLOAD) { st = S_WAIT_START; break; }  // 防溢出
        payloadGot = 0; crcBytes = 0; crcGot = 0;
        st = payloadLen ? S_PAYLOAD : S_CRC;
        break;
    case S_PAYLOAD:
        payload[payloadGot++] = b;
        if (payloadGot >= payloadLen) st = S_CRC;
        break;
    case S_CRC:
        crcGot |= (uint32_t)b << (8 * crcBytes);
        if (++crcBytes == 4) st = S_END;
        break;
    case S_END:
        if (b == 0x03) {
            uint32_t calc = payloadLen ? crc32_calc(payload, payloadLen) : 0;
            if (payloadLen == 0 || calc == crcGot) {
                protocol_handle(frameType, frameIndex, payload, payloadLen);
            }
```

```
        }
        st = S_WAIT_START;
        break;
    }
}
```

四个容易写错的地方：

1. `payloadLen > MAX_PAYLOAD` 必须复位，否则一个坏帧会把后续全部吞掉。
2. CRC 小端：第 1 字节是最低位。
3. 载荷长度 0 的帧（所有 GET 类指令）直接跳 `S_CRC`，此时 CRC 覆盖帧头而不是空载荷——见 8.2。
4. 校验失败要**静默丢弃**并复位，不要试图"跳过一字节重找帧头"，那会连锁错位。

8.8 应答实现

```
void sendAck(uint8_t type) {
    uint8_t f[11] = {0x02, type, 0x00, 0x01, 0x00, 0x00};
    uint32_t c = crc32_calc(f, 6);
    f[6] = c & 0xFF; f[7] = (c >> 8) & 0xFF;
    f[8] = (c >> 16) & 0xFF; f[9] = (c >> 24) & 0xFF;
    f[10] = 0x03;
    SerialBT.write(f, 11);
}

void sendData(uint8_t type, const uint8_t *data, uint16_t len) {
    uint8_t hdr[5] = {0x02, type, 0x00, (uint8_t)(len & 0xFF), (uint8_t)(len >> 8)};
    SerialBT.write(hdr, 5);
    if (len) SerialBT.write(data, len);
    uint32_t c = crc32_calc(data, len);
    uint8_t tail[5] = {(uint8_t)(c), (uint8_t)(c >> 8), (uint8_t)(c >> 16), (uint8_t)(c >> 24),
0x03};
    SerialBT.write(tail, 5);
}
```

发送时一定要检查 `availableForWrite()`：

```
void btSendSafe(const uint8_t *buf, size_t len) {
    size_t sent = 0;
    while (sent < len) {
        size_t can = SerialBT.availableForWrite();
        if (can == 0) { vTaskDelay(1); continue; }
        size_t n = min(can, len - sent);
        SerialBT.write(buf + sent, n);
        sent += n;
    }
}
```

否则大包会被截断——这是"打印出来最后几行缺失"的经典原因。

8.9 常见故障对照

现象	原因	排查
App 显示"连接失败"	握手某条没回 / CRC 错	串口打印收到的 type，逐条比对 8.5

现象	原因	排查
连上但打印无反应	收到 PRINT_DATA 但没触发打印	检查 FEED_LINE(26) 是否收到、是否调用了打印函数
打印内容错位	字节序错 / 每行不足 48 字节	校验 5.3 打包规则
打印一半停	缓冲溢出 / 看门狗	检查 `write` 返回值，加大 `vTaskDelay`
图像上下颠倒	行序反了	从最后一包开始打，或反转行数组
全黑或全白	二值化阈值反了	检查 bit=1 是黑还是白

8.10 自己写上位机

如果不想依赖喵喵机 App，可以自己写。Android 用 `BluetoothSocket` + SPP UUID；Windows 用 Python 的 `pybluez` 或 `pyserial`（先配对一个 COM 口）；Linux 用 `rfcomm`。

流程就是复刻 8.5 的握手 + 8.6 的打印流程。写上位机是理解协议最快的方式——比读文档快十倍。

第 9 章 固件整合

9.1 分层架构

把代码分成三层，是为了"改一处不牵动全身"：

```
┌──────────────────────────────────────┐
│  app 层：业务逻辑                     │  main.cpp / app.cpp
│  - 状态机调度、按键、电量、关机        │
├──────────────────────────────────────┤
│  protocol 层：协议解析与应答          │  protocol.cpp
│  - 帧解析、指令分发、组包应答          │
├──────────────────────────────────────┤
│  hal 层：硬件抽象                     │  printhead.cpp / motor.cpp / adc.cpp
│  - SPI、GPIO、ADC、延时               │
└──────────────────────────────────────┘
```

规则：上层可以调下层，下层不能调上层。`protocol.cpp` 里不允许出现 `digitalWrite`；`printhead.cpp` 里不允许出现 `SerialBT`。这样换硬件只改 hal，换协议只改 protocol。

9.2 config.h：所有"魔法数字"的家

```
#pragma once

// ---------- 打印头 ----------
#define PIN_DI        13
#define PIN_CLK       15
#define PIN_LAT       17
static const int STB_PINS[6] = {14, 27, 26, 25, 33, 32};
#define PIN_VHEN      28

#define HEAT_TIME_BASE   5000     // μs @ density 64
#define HEAT_TIME_MIN     800
#define HEAT_TIME_MAX    8000
#define DOTS_PER_ROW      384
#define BYTES_PER_ROW      48

// ---------- 电机 ----------
#define PIN_MOTOR_AP  23
#define PIN_MOTOR_AM  22
#define PIN_MOTOR_BP  21
#define PIN_MOTOR_BM  19
#define MOTOR_STEP_US 500         // 每拍延时
#define STEPS_PER_ROW   8         // 每行拍数（需按 6.4 标定）

// ---------- 传感 ----------
#define PIN_PHINT     35          // 缺纸，高=缺纸
#define PIN_BATT_ADC  34
#define PIN_TEMP_ADC  36
#define BATT_DIVIDER  6.6f        // 1:1 分压 → ×6.6/4095
#define NTC_R25       10000.0f    // ⚠ 按实际打印头修改
#define NTC_B         3950.0f
#define NTC_R_PULL    10000.0f
```

```
// ---------- 人机 ----------
#define PIN_BTN        5
#define PIN_LED       18
#define PIN_BUZZER    18          // 与 LED 共用时用 LEDC

// ---------- 蓝牙 ----------
#define BT_NAME       "MaoPaperang"
#define PRINTER_SN    "P1001705253855"
#define PRINTER_VER   0x01, 0x00, 0x02

// ---------- 缓冲 ----------
#define MAX_PAYLOAD   2048
#define ROW_BUFFER    64          // 最多缓存 64 行 = 3072 字节
```

9.3 main.cpp：一个能跑的主循环

```
#include <Arduino.h>
#include <SPI.h>
#include "config.h"
#include "printhead.h"
#include "motor.h"
#include "adc.h"
#include "protocol.h"
#include "bluetooth.h"

static uint32_t lastLed = 0;
static uint32_t lastBatt = 0;
static uint8_t  density = 64;

void setup() {
    Serial.begin(115200);
    delay(200);

    // 顺序很重要：先把功率脚拉低，再初始化其它
    pinMode(PIN_VHEN, OUTPUT);
    digitalWrite(PIN_VHEN, HIGH);      // 软开关自锁（若 IO28 兼作自锁）
    stbAll(LOW);
    motorStop();

    printhead_init();
    adc_init();
    protocol_init();
    bt_init();                         // 最后启动蓝牙，此时内存已分配好

    Serial.printf("[SYS] boot ok, free heap=%u\n", ESP.getFreeHeap());
}

void loop() {
    // 1. 缺纸检测（每 100ms）
    static uint32_t lastPaper = 0;
    if (millis() - lastPaper > 100) {
        lastPaper = millis();
        if (paperIsOut()) {
            stbAll(LOW);
            digitalWrite(PIN_VHEN, LOW);
            Serial.println("[PAPER] out");
        }
    }

    // 2. 蓝牙收包
```

```
    bt_poll();

    // 3. 蓝牙发不出去的应答在这里补发
    protocol_flush();

    // 4. 电量（每 5s）
    if (millis() - lastBatt > 5000) {
        lastBatt = millis();
        uint8_t pct = batteryPercent();
        if (pct < 10) Serial.println("[BATT] low");
    }

    // 5. 心跳灯
    if (millis() - lastLed > 1000) {
        lastLed = millis();
        digitalWrite(PIN_LED, !digitalRead(PIN_LED));
    }

    // 6. 空闲喂狗
    vTaskDelay(1);
}
```

9.4 打印任务：队列 + 任务

打印必须独立成任务，否则蓝牙收包一多，STB 关闭就会晚几百微秒，打印出现竖条。

```
struct PrintJob {
    uint16_t rows;
    uint8_t  density;
    uint8_t *data;        // 指向堆上的行缓冲
};

QueueHandle_t printQueue;

void printTask(void *arg) {
    PrintJob job;
    for (;;) {
        if (xQueueReceive(printQueue, &job, portMAX_DELAY) == pdTRUE) {
            printRows(job.data, job.rows, job.density);
            free(job.data);
            Serial.printf("[PRINT] done %u rows\n", job.rows);
        }
    }
}

void print_start() {
    xTaskCreatePinnedToCore(printTask, "print", 8192, NULL, 5, NULL, 1);
}
```

`xTaskCreatePinnedToCore` 的最后一个参数 `1` 表示固定在 core 1，把 core 0 留给蓝牙和 WiFi 协议栈。

9.5 自动关机

协议里有 `SET_POWER_DOWN_TIME(30)`，App 会下发"闲置多久自动关机"（默认 3600 分钟，实际产品里这个字段常被忽略，因为它太长）。

自己实现一个合理的策略：

```
static uint32_t lastActivity = 0;
#define IDLE_SHUTDOWN_MS (10UL * 60 * 1000)   // 10 分钟

void app_touch() { lastActivity = millis(); }

void app_checkIdle() {
    if (SerialBT.hasClient()) { app_touch(); return; }   // 连着就不关
    if (millis() - lastActivity > IDLE_SHUTDOWN_MS) {
        Serial.println("[SYS] shutdown");
        stbAll(LOW);
        motorStop();
        digitalWrite(PIN_VHEN, LOW);     // 若是自锁脚，这一步直接断电
        delay(100);
        esp_deep_sleep_start();          // 兜底：进深睡
    }
}
```

注意：如果 `PIN_VHEN` 兼作整机自锁，拉低它的瞬间就断电了，后面的代码不会执行。这是硬件设计的耦合，改硬件时应该分开（见 2.10）。

9.6 看门狗与异常

```
// 打印过程中如果某段卡住超过 10 秒，强制复位
#include "esp_task_wdt.h"

void printRows(...) {
    esp_task_wdt_add(NULL);
    ...
    for (...) {
        esp_task_wdt_reset();     // 每行喂一次
        ...
    }
    esp_task_wdt_delete(NULL);
}
```

崩溃日志解码：`monitor_filters = esp32_exception_decoder` 会把地址翻译成函数名，直接告诉你哪一行空了指针。

9.7 固件升级的两条路

方式	协议支持	难度	适用
`FIRMWARE_DATA(2)` 分包传入	✅ App 支持	高（要写 Bootloader 交互）	量产
串口重新烧录	❌	低	开发、维修

本项目建议先用串口烧录，把 `FIRMWARE_DATA` 直接回 ACK 忽略掉。App 不会强制升级。

第 10 章 联调与测试

10.1 分阶段测试计划

按这个顺序做，能省掉 80% 的返工：

阶段	测什么	通过标准	工具
P1	电源	3V3=3.30±0.1，VH=8.4±0.3，静态 <60mA	万用表
P2	主控	串口能打印启动日志，心跳灯闪	串口助手
P3	打印头静态	STB 全低，LAT 有脉冲	逻辑分析仪
P4	单行打印	打一条全黑横线，无白点	目视
P5	电机	空转平稳，正反转可控	目视
P6	整页打印	打测试图，比例正确	卡尺
P7	传感器	抽纸报缺纸，电池读数误差 <0.1V	万用表
P8	蓝牙	手机能搜到、能连、回显	手机
P9	协议	App 显示"连接成功"	喵喵机 App
P10	打印	App 打一段文字，清晰、无错位	App
P11	稳定性	连续打 50 页不重启、不缺行	计时

10.2 打印测试页

```
void print_test_page() {
    uint8_t row[48];

    // 全黑
    memset(row, 0xFF, 48);
    printRows(row, 10, 64);

    // 交替竖线（测点位对齐）
    for (int i = 0; i < 48; i++) row[i] = 0xAA;
    printRows(row, 10, 64);

    // 棋盘格（测行距）
    for (int r = 0; r < 20; r++) {
        memset(row, (r % 2) ? 0xAA : 0x55, 48);
        printRows(row, 1, 64);
    }
}
```

三段图案分别验证：**加热均匀性**（全黑有无白点）、**点位对齐**（竖线是否笔直）、**行距**（棋盘格是否正方形）。

10.3 时序验证

有逻辑分析仪的话，抓四根线：CLK、DI、LAT、STB1。

要确认的三件事：

1. **LAT 脉冲在 DI 传完之后**：如果 LAT 提前，会锁存半行数据。
2. **STB 宽度 = heat_us**：实测应与你算的 5000µs 一致（±5%）。
3. **6 个 STB 是否重叠**：如果同时拉高，峰值电流是 6 倍，电源会被拉垮。

没有逻辑分析仪时，用"打一条细横线看深浅"也能间接判断：如果线中间比两端深，说明 VH 压降大。

10.4 电源完整性

打印瞬间用示波器看 VH 和 3V3：

现象	判断	处理
VH 跌到 7V 以下	输出电容不够 / 电池内阻大	加 100µF，换低内阻电池
3V3 跌到 3.0V 以下	LDO 限流或去耦不足	加大 3V3 电容，检查 LDO 额定电流
出现 >500mV 尖峰	缺少吸收	VH 端加 TVS，STB 串 22Ω

判定标准：VH 跌落 <5%、3V3 跌落 <3%，否则打印会出现"越打越淡"。

10.5 长时间稳定性

"打 50 页不重启"是量产门槛。常见失败原因：

- 内存泄漏（`String` 拼接、`new` 不 `delete`）→ 每页打印 `ESP.getFreeHeap()`，看是否单调下降。
- 温度累积 → 打印头温度 >75℃ 时降速，否则会糊。
- 电池电压下降 → 低电量时自动降 density。

```
// 每页打印后记录
Serial.printf("[STAT] page=%d heap=%u vh=%dmV\n",
              pageCount, ESP.getFreeHeap(), analogReadMilliVolts(34) * 2);
```

10.6 常见问题总表

#	现象	最可能原因	解决方案
1	编译报 `BluetoothSerial.h` 找不	Arduino-ESP32 3.x	锁 `espressif32@6.5.0`

#	现象	最可能原因	解决方案
	到		
2	烧录失败	未进下载模式/串口占用	手动 BOOT+EN
3	上电不启动	EN 脚 RC 不对 / 3V3 不足	加 0.1μF，量 3V3
4	按开机键没反应	软开关 MOS 接反 / 自锁 IO 没拉高	查 Q21/Q23 极性
5	打印全白	VH 没开 / STB 没拉高	量 VH，量 STB
6	打印全黑	数据全 1 / 加热过长	检查打包，降 density
7	有竖白条	某段 STB 断 / 该点坏	查 FPC，换头
8	越打越淡	电池跌压 / 温度高	降密度，加电容
9	图像纵向拉长	行距不对	按 6.4 标定
10	图像横向错位	每行不足 48 字节	补齐 0
11	电机不转	相序错 / 没通电	万用表分相
12	电机抖动	拍间延时不均	用 `micros` 校准
13	电机发烫	停止未断电	四脚拉低
14	一直报缺纸	传感器脏 / 阈值偏	清洁，调分压
15	电量跳变	ADC 噪声 / 未滤波	多次采样 + 0.1μF
16	温度读数离谱	NTC 参数不符	按实际改 R25/B
17	手机搜不到	已配对旧记录	忽略设备重搜
18	连上就断	内存不足	减缓冲，查 heap
19	App 连接失败	握手漏指令/CRC 错	逐条比对 8.5
20	打印中途停	发送缓冲溢出	检查 `write` 返回值
21	打一半重启	欠压/看门狗	看 `esp_reset_reason()`
22	蓝牙距离短	天线净空破坏	清空天线区

10.7 性能数据（实测参考）

指标	数值	条件
单行耗时	9.4ms	heat 5ms + motor 4ms + SPI 0.4ms
打印速度	约 13mm/s	8 行/mm
一页（100mm）	约 8 秒	800 行
蓝牙吞吐	52KB/s	1920B/包
待机电流	45mA	蓝牙连接、灯闪
打印峰值	3.2A	6 段同时
续航	40 分钟连续打印	2000mAh

> 如果嫌慢，最有效的优化是**加热与走纸重叠**（STB 拉低后立刻启动电机，不等走完再传下一行），能把单行从 9.4ms 压到 5.4ms，速度提升 70%。代价是时序更紧，需要 `micros()` 精确控制。

附录

附录 A 引脚总表

GPIO	功能	方向	备注
1	U0TXD	出	调试串口
3	U0RXD	入	调试串口
5	BTN	入	上拉，按下为低
12	DI	出	打印头数据
13	DI（示例代码）/ 见注	出	以实际工程为准
14	STB1	出	第 1 段
15	CLK	出	打印头时钟
17	LAT	出	锁存
18	LED	出	低电平点亮
19	MOTOR_BM (B+)	出	H 桥 IN4
21	MOTOR_BP (B-)	出	H 桥 IN3
22	MOTOR_AM (A+)	出	H 桥 IN2
23	MOTOR_AP (A-)	出	H 桥 IN1
25	STB4	出	
26	STB3	出	
27	STB2	出	
28	VH_EN	出	高=开打印头电源（兼自锁）
32	STB6	出	
33	STB5	出	
34	BATT_ADC	入	仅输入，1:1 分压
35	PHINT	入	仅输入，高=缺纸
36	THER	入	仅输入，NTC

> 注：示例代码里 `main.h` 定义 `PIN_DI 13`、`PIN_CLK 15`，而原理图里 IO12 与 IO13 的标注存在冲突，实际以你的板子为准——**先用万用表确认 FPC 座的 DI/CLK 实际连到哪个 IO**，再改 `config.h`。

附录 B 指令速查卡

```
握手：24 → 10 → 37/66 → 127 → 25 → 48 → 16 → 31 → 4 → 44
打印：25 ×2 → 26 → 0 ×N（≤1920B）→ 26
查询：4 版本 / 10 SN / 16 电量 / 18 温度 / 28 密度 / 31 关机时间 / 48 设备名
帧：02 | type | idx | lenL lenH | payload | crc32(LE) | 03
```

```
ACK: 02 | type | 00 | 01 00 | 00 | crc32(前 6 字节) | 03
```

附录 C 物料清单（BOM）

类别	型号	数量	备注
主控	ESP32-WROOM-32E	1	4MB Flash
打印头	JX-2R-01 / JX-700-48 兼容	1	384 点，含 NTC
电机	两相四线步进	1	9°/步
电机驱动	TC1508S	1	双 H 桥
升压	AP2005（或同类）	1	8.4V 输出
电感	4.7μH/3A	1	低 DCR
二极管	SS54	1	40V/5A
LDO	XC6210B332MR	1	3.3V/500mA
充电	ME4054	1	500mA
比较器	LM393	1	缺纸检测
P-MOS	AO4407A	1	电源自锁
N-MOS	AO3400A	1	自锁驱动
USB 转串口	CH340C	1	自动下载
Type-C	16P 母座	1	仅充电+数据
FPC 座	0.5mm 32P	1	打印头
电池	18650 2000mAh 带保护	1	≥5A 放电
电容	47μF×2、22μF、10μF×3、0.1μF×10	—	低 ESR

附录 D 术语表

术语	解释
热敏打印	加热使热敏纸涂层变色，无需墨
STB	Strobe，选通信号，控制哪一段加热
LAT	Latch，锁存信号，把移位数据搬进锁存器
SPP	Serial Port Profile，经典蓝牙的串口仿真
RFCOMM	基于 L2CAP 的串口复用协议
CRC32	32 位循环冗余校验
半步/八拍	单相与双相交替通电，步距减半
density	热密度，控制加热时间
丢步	电机实际走的步数少于指令步数

术语	解释
欠压复位	电源跌落到阈值以下触发重启

附录 E 参考资料

1. 《蓝牙打印机开发资料-仅供参考》（本项目资料 6，45 页）
2. 《JX-2R-01 打印头规格书》（资料 1）
3. 《ESP32-WROOM-32E 参考手册》《ESP32 中文技术手册》（资料 1）
4. 原理图 `SCH_Schematic4_6_2025-10-17.pdf`（资料 5）
5. 示例代码（资料 7，Arduino 框架）
6. 喵喵机 App 指令合集 `const.txt` / `const_newest.txt`（资料 3）
7. Espressif Arduino-ESP32 文档：https://docs.espressif.com/projects/arduino-esp32/
8. PlatformIO 文档：https://docs.platformio.org/

附录 F 出厂前检查清单

- [ ] 3V3 实测 3.3V±0.1，纹波 <50mV
- [ ] VH 实测 8.4V±0.3，打印时跌落 <5%
- [ ] 静态电流 <60mA，关机电流 <50μA
- [ ] STB 上电默认为低（用示波器看复位期间）
- [ ] 缺纸时 STB 立即拉低、VH 关闭
- [ ] 打印 100 行方块，高度误差 <2%
- [ ] 全黑页无竖白条
- [ ] 连续 50 页不重启
- [ ] 手机 10 米内可连接
- [ ] App 握手成功、能打印文字与图片
- [ ] 电量显示与实际电压误差 <0.1V
- [ ] 温度显示与室温误差 <3℃
- [ ] 长按按键能关机，短按能走纸
- [ ] 充电指示灯：充电亮、充满灭
- [ ] 天线净空区无铜、无电池